Alan Samuel Gomez Paz - 22115
Samuel David Tórtola Bardales - 22094
Luis Alejandro Luna Hernández - 22017

Gestor de Monitoreo de Red

NETDATA

Guía para conexión (monitoreo) de equipos a su Gestor + Gestión de una Red MPLS



UVG
UNIVERSIDAD DEL VALLE DE GUATEMALA

Índice general

1. Introducción **3**

2. Configuración y Uso de SNMP en huawei **4**
2.1. Conexión entre la Raspberry Pi y el Router 4
2.2. Configuración de SNMP en el Router 5
2.3. Configuración de Netdata para recibir al router 7

3. Configuración y puesta en marcha de la red MPLS **9**
3.1. Topología de la red MPLS 9
3.1.1. Configuración base y OSPF 12
3.1.2. Configuración del LSP estático 15
3.1.3. Eliminación del LSP estático 17
3.1.4. Configuración de LDP 18
3.1.5. Cambio del modo de anuncio a DoD 20
3.1.6. Comandos de diagnóstico 21
3.1.7. Almacenamiento de la configuración 22
3.1.8. Limpieza final 22
3.1.9. Configuración del acceso SSH 22
3.2. Generación y verificación de tráfico sobre MPLS 23
3.2.1. Direccionamiento de las redes de prueba 23
3.2.2. Configuración de la red de acceso en AR651-A 24
3.2.3. Configuración de la red de acceso en AR651-B 24
3.2.4. Configuración de las computadoras 25
3.2.5. Creación de LSP para las redes de acceso 26
3.2.6. Verificación del etiquetado MPLS 27
3.2.7. Instalación y preparación de iPerf3 27
3.2.8. Pruebas de rendimiento con TCP 28
3.2.9. Pruebas de rendimiento con UDP 28
3.2.10. Criterios de aceptación de la prueba 29

4. Configuración y automatización con Netdata **30**
4.1. Creación de Dashboards en NetData 30
4.2. Editar Dashboards en NetData 32
4.3. Configuración de una alerta de latencia para los dispositivos SNMP 34
4.4. Configuración de alertas por fallo en la recolección SNMP 36
4.4.1. Prueba de funcionamiento 38
4.5. Configuración de la topología SNMP en Netdata 39
4.5.1. Activación del recolector de topología 39
4.5.2. Creación del directorio de perfiles personalizados 39
4.5.3. Creación del perfil genérico personalizado 40
4.5.4. Creación del perfil de topología para Huawei 40

4.5.5. Asignación de propietarios y permisos . . . 41
4.5.6. Resolución local de los nombres de los dispositivos . . . 41
4.5.7. Aplicación y verificación de la configuración . . . 42
4.6. Implementación de una topología de red personalizada . . . 43
4.6.1. Arquitectura de la solución . . . 43
4.6.2. Contenido . . . 44
4.6.3. Topología representada . . . 44
4.6.4. Métricas utilizadas . . . 44
4.6.5. Funcionamiento del servidor Python . . . 46
4.6.6. Actualización automática de la página . . . 46
4.6.7. Instalación de la aplicación . . . 46
4.6.8. Creación del servicio de systemd . . . 47
4.6.9. Acceso a la topología . . . 48
4.6.10. Consideraciones de seguridad . . . 48
4.7. Habilitación del monitoreo de OSPF y MPLS mediante SNMP . . . 48
4.7.1. Vinculación del proceso OSPF con SNMP . . . 48
4.7.2. Importancia de la vinculación OSPF . . . 49
4.7.3. Estados de los vecinos OSPF . . . 49
4.7.4. Consulta del estado MPLS LDP . . . 50
4.7.5. Estados de las sesiones MPLS LDP . . . 51
4.7.6. Interpretación conjunta de los estados . . . 51
4.7.7. Crear el perfil de enrutamiento en Netdata . . . 52

5. Anexo **65**
5.1. Anexo . . . 65

Capítulo 1

Introducción

La segunda fase del proyecto tiene como propósito integrar equipos de red al gestor de monitoreo Netdata instalado en la Raspberry Pi y aplicar posteriormente los principios de gestión sobre una red MPLS. La Raspberry Pi funcionará como el punto central de recolección de métricas, mientras que Netdata Cloud proporcionará la interfaz centralizada para que los integrantes del equipo accedan mediante cuentas individuales, consulten el estado de la infraestructura y utilicen los tableros de monitoreo.

La conexión de los dispositivos dependerá de sus características. En servidores compatibles se podrá instalar un agente de Netdata para obtener información detallada del sistema operativo, como el uso del procesador, memoria, almacenamiento, interfaces y servicios. En equipos de red, como switches y routers Huawei, se utilizará el protocolo SNMP, el cual permite que un sistema de gestión consulte variables operativas almacenadas en la base de información de administración o MIB del dispositivo.

Para la primera parte se configurará un equipo Huawei con una dirección IP de administración y SNMPv3. Esta versión proporciona mecanismos de autenticación y cifrado, ofreciendo mayor seguridad que las versiones anteriores del protocolo [1, sec. 1.3]. Después se configurará Netdata para consultar el equipo desde la Raspberry Pi y presentar métricas como el estado y tráfico de las interfaces, errores, tiempo de actividad y utilización de recursos, según la información que el modelo del dispositivo permita obtener mediante SNMP.

La segunda parte extenderá el procedimiento hacia una red MPLS compuesta por routers Huawei AR651 y switches Huawei S6730H. Antes de implementar MPLS se establecerá conectividad IP entre todos los dispositivos y se utilizará OSPF como protocolo de enrutamiento interno. Posteriormente se configurarán y verificarán caminos conmutados por etiquetas, inicialmente de manera estática y después mediante LDP, observando las operaciones de inserción, intercambio y eliminación de etiquetas realizadas por los equipos de borde y tránsito [2, partes 1–4].

De esta manera, el trabajo abarcará dos niveles de gestión. En el primero se incorporará y supervisará un equipo Huawei individual mediante SNMPv3. En el segundo se implementará una topología MPLS completa y se integrarán sus dispositivos al sistema de monitoreo. Esto permitirá relacionar la configuración lógica de la red con la observación centralizada de su disponibilidad, rendimiento e interfaces.

Capítulo 2

Configuración y Uso de SNMP en huawei

2.1. Conexión entre la Raspberry Pi y el Router

Para que la Raspberry Pi pueda tener comunicación con el Router, se procedió a colocar una ip fija en la Raspberry Pi, para ello se utilizó el siguiente comando:

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

Contenido del archivo 50-cloud-init.yaml:

```
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: no
      dhcp6: no
      addresses:
        - 192.168.10.1/24
```

Luego de esto, se procedió de igual manera configurando una ip en el puerto del router que se encuentra conectado a la Raspberry Pi, siguiendo el procedimiento de conectividad entre el router y el gestor de red [1, sec. 1.4.2, paso 1]. Para ello se utilizó el siguiente comando:

```
<Huawei>system-view
[Huawei]interface GigabitEthernet 0/0/0
[Huawei-GigabitEthernet0/0/0]undo portswitch
[Huawei-GigabitEthernet0/0/0]ip address 192.168.10.2 255.255.255.0
[Huawei-GigabitEthernet0/0/0]quit
```

En el caso de los switches del laboratorio, se utilizó el siguiente comando, para la interfaz de entrada de SNMP:

```
snmp-agent protocol source-interface LoopBack 0
```

2.2. Configuración de SNMP en el Router

La habilitación del agente, la definición de la vista MIB y la creación del grupo y del usuario SNMPv3 se basan en la guía de gestión de dispositivos de Huawei [1, sec. 1.4.2], con los parámetros adaptados al monitoreo de lectura desde Netdata. La referencia de comandos SNMP de Huawei complementa la consulta de la sintaxis y de las opciones de configuración [3].

- Se activa SNMP

```
<Huawei>system-view
[Huawei]sysname Router-Monitoreado
[Router-Monitoreado]snmp-agent permit interface GigabitEthernet0/0/0
[Router-Monitoreado]snmp-agent
[Router-Monitoreado]snmp-agent sys-info version v3
```

  Para monitorear interfaces, tráfico, información del sistema y otros recursos, se crea una vista.

```
[Router-Monitoreado]snmp-agent mib-view netdata-view include iso
```

  En el caso de los switches del laboratorio, el comando iso es:

```
snmp-agent mib-view included netdata-view iso
```

  El parámetro `include` indica que el subárbol especificado será incorporado dentro de la vista permitida, mientras que `iso` representa el nodo raíz del árbol de identificadores de objetos SNMP, equivalente al OID `1`. Por lo tanto, esta configuración permite que Netdata consulte tanto las MIB estándar, utilizadas para obtener información del sistema, interfaces, tráfico, errores y tiempo de actividad, como las MIB empresariales propias de Huawei.

- Crear el grupo SNMPv3

```
[Router-Monitoreado]snmp-agent group v3 netdata-group privacy read-
    view netdata-view
```

  - v3: utiliza SNMPv3.
  - privacy: exige autenticación y cifrado.
  - read-view netdata-view: concede acceso de lectura a la vista creada.
  - No se configura write-view, porque Netdata solamente necesita consultar información.

- Crear el usuario SNMPv3 y asignarlo al grupo creado anteriormente.

```
[Router-Monitoreado]snmp-agent usm-user v3 netdata-user group netdata-
    group
```

```
[Router-Monitoreado]snmp-agent usm-user v3 netdata-user authentication
    -mode sha2-256
```

El router pedira contraseña para el usuario, se ingresa la contraseña deseada y se confirma. En este caso se utilizó la contraseña `NetAuth2026`.

```
[Router-Monitoreado1]snmp-agent usm-user v3 netdata-user privacy-mode
    aes128
```

El router pedira contraseña para el usuario, se ingresa la contraseña deseada y se confirma. En este caso se utilizó la contraseña `NetPriv2026`.

- Comprobar la configuración de SNMPv3 con el siguiente comando, conforme a la verificación de usuarios indicada en la guía [1, sec. 1.5, paso 2]:

```
[Router-Monitoreado]display snmp-agent usm-user

[Router-Monitoreado]quit
<Router-Monitoreado>save
```

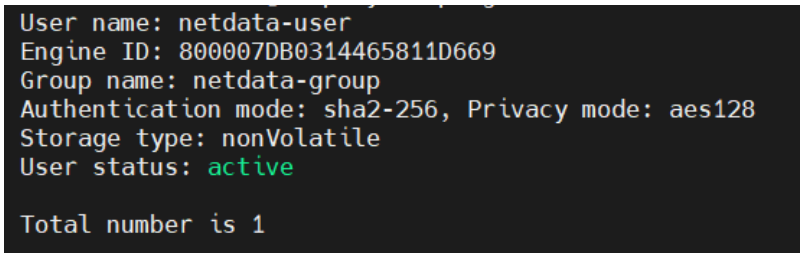


Figura 2.1: Confirmación de configuración de SNMPv3 en el Router

- Para comprobar que la Raspberry Pi puede comunicarse con el Router, se utilizó el siguiente comando:

  Primero se deben de instalar los paquetes necesarios para poder utilizar el comando `snmpwalk`:

```
sudo apt update
sudo apt upgrade
sudo apt install -y snmp
```

```
snmpwalk -v3 \
  -l authPriv \
  -u netdata-user \
  -a SHA-256 \
  -A ’NetAuth2026’’ \
  -x AES \
  -X ’NetPriv2026’ \
  192.168.10.2 \
  1.3.6.1.2.1.1
```

```
iso.3.6.1.2.1.1.1.0 = STRING: "Huawei AR651 Huawei Versatile Routing Platform Software
 2011-2022 Huawei Technologies Co., Ltd"
iso.3.6.1.2.1.1.2.0 = OID: iso.3.6.1.4.1.2011.2.224.323
iso.3.6.1.2.1.1.3.0 = Timeticks: (353133) 0:58:51.33
iso.3.6.1.2.1.1.4.0 = STRING: "R&D Shenzhen, Huawei Technologies Co., Ltd."
iso.3.6.1.2.1.1.5.0 = STRING: "Router-Monitoreado1"
iso.3.6.1.2.1.1.6.0 = STRING: "Shenzhen China"
iso.3.6.1.2.1.1.7.0 = INTEGER: 78
```

Figura 2.2: Confirmación de lectura de información del Router desde la Raspberry Pi

2.3. Configuración de Netdata para recibir al router

La configuración del colector utiliza un trabajo por dispositivo y los campos de autenticación y cifrado SNMPv3 descritos en la documentación de Netdata [4].

- El archivo de configuración de SNMP se encuentra en la siguiente ruta: `/etc/netdata/go.d/snmp.conf`. Para editarlo se utiliza el siguiente comando:

```
sudo nano /etc/netdata/go.d/snmp.conf
```

- Se utiliza el siguiente contenido para el archivo de configuración de SNMP:

```
jobs:
  - name: huawei_router
    hostname: 192.168.10.2
    update_every: 60
    timeout: 5

    options:
      version: 3

    user:
      name: netdata-user
      level: authPriv
      auth_proto: sha256
      auth_key: "NetAuth2026"
      priv_proto: aes
      priv_key: "NetPriv2026"
```

  Si se desea monitorear más de un router, se puede agregar otro bloque de configuración con el mismo formato, cambiando el nombre del trabajo y la dirección IP del router correspondiente.

- Luego de cerrar el archivo, se colocó seguridad de que el archivo de configuración de SNMP tenga los permisos correctos, para ello se utilizó el siguiente comando:

```
sudo chown root:netdata /etc/netdata/go.d/snmp.conf
sudo chmod 640 /etc/netdata/go.d/snmp.conf
```

- Luego de estos pasos, se debe de reiniciar el servicio de Netdata para que los cambios surtan efecto, para ello se utilizó el siguiente comando:

```
sudo systemctl restart netdata
```

- Al ingresar a la interfaz web de Netdata, se puede observar que el router se encuentra monitoreado y se puede visualizar información como tráfico, errores, tiempo de actividad, entre otros.



Figura 2.3: Interfaz web de Netdata mostrando información del router monitoreado

Estos mismos pasos se realizan para todos los demás routers que se deseen monitorear, cambiando únicamente la dirección IP del router y el nombre del router y el nombre del trabajo en el archivo de configuración de SNMP. (El nombre que muestra Netdata es el nombre del router que se configuró en el router, en este caso `Router-Monitoreado`).

Capítulo 3

Configuración y puesta en marcha de la red MPLS

3.1. Topología de la red MPLS

La figura 5.1 presenta la topología de la red MPLS utilizada en el proyecto. En esta topología, se puede observar la interconexión entre los routers y la Raspberry Pi, que actúa como un nodo de monitoreo para capturar y analizar el tráfico de la red mediante SNMP y Netdata.

El núcleo MPLS, los roles de los cuatro equipos y el plan de direccionamiento se basan en la guía de laboratorio para routers AR651 y switches S6730H [2, parte 0]. La Raspberry Pi y la red de monitoreo se incorporan a esa base para este proyecto.

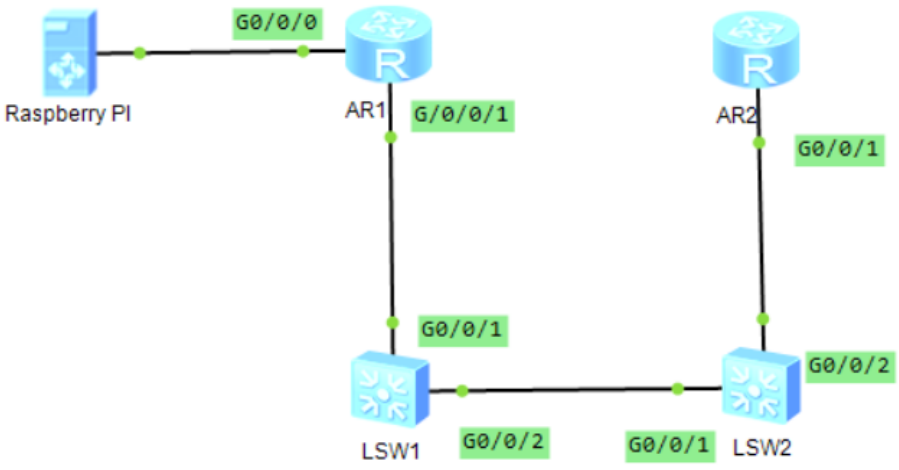


Figura 3.1: Topologia de la red MPLS

Para que el servidor de monitoreo, en este caso la Raspberry Pi, pueda capturar y analizar el tráfico de la red, es necesario que pueda tener comunicación con todos los dispositivos de la red, para esto se configuró rutas estáticas en la Raspberry Pi, de tal manera que pueda alcanzar a todos los dispositivos de la red MPLS. Para eso, se modificó el archivo: sudo nano /etc/netplan/50-cloud-init.yaml

Contenido del archivo:

```
network:
  version: 2
```

```
  ethernets:
    eth0:
      dhcp4: false
      dhcp6: false
      addresses:
        - 192.168.10.1/24
      routes:
        - to: 1.1.1.1/32
          via: 192.168.10.2
        - to: 2.2.2.2/32
          via: 192.168.10.2
        - to: 3.3.3.3/32
          via: 192.168.10.2
        - to: 4.4.4.4/32
          via: 192.168.10.2
```

Luego, se confirma la configuración de la red con el comando:

```
sudo netplan try
```

La Raspberry Pi, probará la configuración de red y si todo está correcto, se aplicará la configuración de manera permanente, al presionar Enter. En caso de que la configuración no sea correcta, se revertirá a la configuración anterior.

Ahora, se debe de configurar netdata, para esto, se modifica el archivo respectivo de netdata, com se realizó en el caítulo 2. La definición de varios trabajos en el archivo de configuración sigue el esquema de un trabajo por dispositivo documentado por Netdata [4].

```
jobs:
  - name: AR651-A
    hostname: 1.1.1.1
    update_every: 10
    create_vnode: true
    vnode_device_down_threshold: 3

    options:
      version: 3
      port: 161
      timeout: 10
      retries: 1
      max_request_size: 20
      max_repetitions: 10

    user:
      name: netdata-user
      level: authPriv
      auth_proto: sha256
      auth_key: "NetAuth2026"
      priv_proto: aes
      priv_key: "NetPriv2026"

    manual_profiles:
      - huawei
      - huawei-routing-status

  - name: S6730H-C
    hostname: 2.2.2.2
```

```
    update_every: 10
    create_vnode: true
    vnode_device_down_threshold: 3

    options:
      version: 3
      port: 161
      timeout: 10
      retries: 1
      max_request_size: 20
      max_repetitions: 10

    user:
      name: netdata-user
      level: authPriv
      auth_proto: sha256
      auth_key: "NetAuth2026"
      priv_proto: aes
      priv_key: "NetPriv2026"

    manual_profiles:
      - huawei
      - huawei-routing-status

  - name: S6730H-D
    hostname: 3.3.3.3
    update_every: 10
    create_vnode: true
    vnode_device_down_threshold: 3

    options:
      version: 3
      port: 161
      timeout: 10
      retries: 1
      max_request_size: 20
      max_repetitions: 10

    user:
      name: netdata-user
      level: authPriv
      auth_proto: sha256
      auth_key: "NetAuth2026"
      priv_proto: aes
      priv_key: "NetPriv2026"

    manual_profiles:
      - huawei
      - huawei-routing-status

  - name: AR651-B
    hostname: 4.4.4.4
    update_every: 10
    create_vnode: true
    vnode_device_down_threshold: 3

    options:
      version: 3
```

```
      port: 161
      timeout: 10
      retries: 1
      max_request_size: 20
      max_repetitions: 10

    user:
      name: netdata-user
      level: authPriv
      auth_proto: sha256
      auth_key: "NetAuth2026"
      priv_proto: aes
      priv_key: "NetPriv2026"

    manual_profiles:
      - huawei
      - huawei-routing-status
```

La implementación utiliza dos routers Huawei AR651 como dispositivos de borde y dos switches Huawei S6730H como dispositivos de tránsito. La configuración se divide en cuatro etapas: configuración del protocolo OSPF, creación de un LSP estático, implementación de LDP y modificación del modo de anuncio de etiquetas.

Antes de ejecutar los comandos se debe comprobar el nombre real de las interfaces:

```
<HUAWEI> display interface brief
```

Los puertos Ethernet de los switches S6730H deben funcionar como interfaces de capa 3. Para ello se utiliza el comando `undo portswitch`. Si el comando no está disponible o la interfaz ya trabaja en capa 3, se puede omitir.

3.1.1. Configuración base y OSPF

La configuración de interfaces, loopbacks y OSPF sigue la parte 1 de la guía MPLS [2, parte 1]. Como antecedentes para la configuración de OSPF de área única se utilizan la guía de operación y mantenimiento de Huawei [5, sec. 1.4.2, pasos 1–2] y el informe del laboratorio 14 [6, pp. 2–7], adaptando el direccionamiento a la topología de este proyecto.

Configuración de AR651-A

```
<HUAWEI> system-view
[HUAWEI] sysname AR651-A

[AR651-A] interface GigabitEthernet0/0/1
[AR651-A-GigabitEthernet0/0/1] ip address 10.0.12.1 255.255.255.252
[AR651-A-GigabitEthernet0/0/1] undo shutdown
[AR651-A-GigabitEthernet0/0/1] quit

[AR651-A] interface LoopBack0
[AR651-A-LoopBack0] ip address 1.1.1.1 255.255.255.255
[AR651-A-LoopBack0] quit

[AR651-A] ospf 1
[AR651-A-ospf-1] area 0.0.0.0
```

```
[AR651-A-ospf-1-area-0.0.0.0] network 1.1.1.1 0.0.0.0
[AR651-A-ospf-1-area-0.0.0.0] network 10.0.12.0 0.0.0.3
[AR651-A-ospf-1-area-0.0.0.0] network 192.168.10.0 0.0.0.255
[AR651-A-ospf-1-area-0.0.0.0] quit
[AR651-A-ospf-1] quit
[AR651-A] return
```

Como se puede observar, se anuncia la red 192.168.10.0, la cual corresponde a la red de monitoreo donde se encuentra la Raspberry Pi, para que pueda tener comunicación con todos los dispositivos de la red MPLS.

Configuración de S6730H-C

```
<HUAWEI> system-view
[HUAWEI] sysname S6730H-C

[S6730H-C] interface GigabitEthernet0/0/1
[S6730H-C-GigabitEthernet0/0/1] undo portswitch
[S6730H-C-GigabitEthernet0/0/1] ip address 10.0.12.2 255.255.255.252
[S6730H-C-GigabitEthernet0/0/1] undo shutdown
[S6730H-C-GigabitEthernet0/0/1] quit

[S6730H-C] interface GigabitEthernet0/0/2
[S6730H-C-GigabitEthernet0/0/2] undo portswitch
[S6730H-C-GigabitEthernet0/0/2] ip address 10.0.23.1 255.255.255.252
[S6730H-C-GigabitEthernet0/0/2] undo shutdown
[S6730H-C-GigabitEthernet0/0/2] quit

[S6730H-C] interface LoopBack0
[S6730H-C-LoopBack0] ip address 2.2.2.2 255.255.255.255
[S6730H-C-LoopBack0] quit

[S6730H-C] ospf 1
[S6730H-C-ospf-1] area 0.0.0.0
[S6730H-C-ospf-1-area-0.0.0.0] network 2.2.2.2 0.0.0.0
[S6730H-C-ospf-1-area-0.0.0.0] network 10.0.12.0 0.0.0.3
[S6730H-C-ospf-1-area-0.0.0.0] network 10.0.23.0 0.0.0.3
[S6730H-C-ospf-1-area-0.0.0.0] quit
[S6730H-C-ospf-1] quit
[S6730H-C] return
```

Configuración de S6730H-D

```
<HUAWEI> system-view
[HUAWEI] sysname S6730H-D

[S6730H-D] interface GigabitEthernet0/0/1
[S6730H-D-GigabitEthernet0/0/1] undo portswitch
[S6730H-D-GigabitEthernet0/0/1] ip address 10.0.23.2 255.255.255.252
[S6730H-D-GigabitEthernet0/0/1] undo shutdown
[S6730H-D-GigabitEthernet0/0/1] quit

[S6730H-D] interface GigabitEthernet0/0/2
[S6730H-D-GigabitEthernet0/0/2] undo portswitch
```

```
[S6730H-D-GigabitEthernet0/0/2] ip address 10.0.34.1 255.255.255.252
[S6730H-D-GigabitEthernet0/0/2] undo shutdown
[S6730H-D-GigabitEthernet0/0/2] quit

[S6730H-D] interface LoopBack0
[S6730H-D-LoopBack0] ip address 3.3.3.3 255.255.255.255
[S6730H-D-LoopBack0] quit

[S6730H-D] ospf 1
[S6730H-D-ospf-1] area 0.0.0.0
[S6730H-D-ospf-1-area-0.0.0.0] network 3.3.3.3 0.0.0.0
[S6730H-D-ospf-1-area-0.0.0.0] network 10.0.23.0 0.0.0.3
[S6730H-D-ospf-1-area-0.0.0.0] network 10.0.34.0 0.0.0.3
[S6730H-D-ospf-1-area-0.0.0.0] quit
[S6730H-D-ospf-1] quit
[S6730H-D] return
```

Configuración de AR651-B

```
<HUAWEI> system-view
[HUAWEI] sysname AR651-B

[AR651-B] interface GigabitEthernet0/0/1
[AR651-B-GigabitEthernet0/0/1] ip address 10.0.34.2 255.255.255.252
[AR651-B-GigabitEthernet0/0/1] undo shutdown
[AR651-B-GigabitEthernet0/0/1] quit

[AR651-B] interface LoopBack0
[AR651-B-LoopBack0] ip address 4.4.4.4 255.255.255.255
[AR651-B-LoopBack0] quit

[AR651-B] ospf 1
[AR651-B-ospf-1] area 0.0.0.0
[AR651-B-ospf-1-area-0.0.0.0] network 4.4.4.4 0.0.0.0
[AR651-B-ospf-1-area-0.0.0.0] network 10.0.34.0 0.0.0.3
[AR651-B-ospf-1-area-0.0.0.0] quit
[AR651-B-ospf-1] quit
[AR651-B] return
```

Verificación de OSPF

Los siguientes comandos deben ejecutarse en los dispositivos para comprobar las adyacencias y las rutas aprendidas mediante OSPF, siguiendo la comprobación de conectividad de la guía [2, tarea 1.3]:

```
<HUAWEI> display ospf peer brief
<HUAWEI> display ip routing-table protocol ospf
```

Desde AR651-A se comprueba la comunicación entre las loopbacks de ambos extremos:

```
<AR651-A> ping -a 1.1.1.1 4.4.4.4
```

No se debe continuar con MPLS hasta obtener respuesta y comprobar que existe una ruta hacia `4.4.4.4/32`:

```
<AR651-A> display ip routing-table 4.4.4.4
```

3.1.2. Configuración del LSP estático

El procedimiento de construcción del LSP estático se basa en la parte 2 de la guía de laboratorio [2, parte 2], con el siguiente plan de etiquetas adaptado para esta implementación:

- AR651-A inserta la etiqueta 100.
- S6730H-C recibe la etiqueta 100 y la sustituye por la etiqueta 200.
- S6730H-D recibe la etiqueta 200 y la sustituye por la etiqueta 300.
- AR651-B recibe y elimina la etiqueta 300.

Esta implementación corresponde a un LSP estático convencional. La guía propone utilizar PHP en el último salto del LSP estático [2, tarea 2.3]; en esta implementación, el comportamiento PHP se observará posteriormente con LDP.

Configuración MPLS de AR651-A

```
<AR651-A> system-view
[AR651-A] mpls lsr-id 1.1.1.1
[AR651-A] mpls
[AR651-A-mpls] quit

[AR651-A] interface GigabitEthernet0/0/1
[AR651-A-GigabitEthernet0/0/1] mpls
[AR651-A-GigabitEthernet0/0/1] quit

[AR651-A] static-lsp ingress LSP-A-B destination 4.4.4.4 32 nexthop
    10.0.12.2 out-label 100
[AR651-A] return
```

Configuración MPLS de S6730H-C

```
<S6730H-C> system-view
[S6730H-C] mpls lsr-id 2.2.2.2
[S6730H-C] mpls
[S6730H-C-mpls] quit

[S6730H-C] interface GigabitEthernet0/0/1
[S6730H-C-GigabitEthernet0/0/1] mpls
[S6730H-C-GigabitEthernet0/0/1] quit

[S6730H-C] interface GigabitEthernet0/0/2
[S6730H-C-GigabitEthernet0/0/2] mpls
[S6730H-C-GigabitEthernet0/0/2] quit
```

```
[S6730H-C] static-lsp transit LSP-A-B incoming-interface GigabitEthernet0
    /0/1 in-label 100 nexthop 10.0.23.2 out-label 200
[S6730H-C] return
```

Configuración MPLS de S6730H-D

```
<S6730H-D> system-view
[S6730H-D] mpls lsr-id 3.3.3.3
[S6730H-D] mpls
[S6730H-D-mpls] quit

[S6730H-D] interface GigabitEthernet0/0/1
[S6730H-D-GigabitEthernet0/0/1] mpls
[S6730H-D-GigabitEthernet0/0/1] quit

[S6730H-D] interface GigabitEthernet0/0/2
[S6730H-D-GigabitEthernet0/0/2] mpls
[S6730H-D-GigabitEthernet0/0/2] quit

[S6730H-D] static-lsp transit LSP-A-B incoming-interface GigabitEthernet0
    /0/1 in-label 200 nexthop 10.0.34.2 out-label 300
[S6730H-D] return
```

Configuración MPLS de AR651-B

```
<AR651-B> system-view
[AR651-B] mpls lsr-id 4.4.4.4
[AR651-B] mpls
[AR651-B-mpls] quit

[AR651-B] interface GigabitEthernet0/0/1
[AR651-B-GigabitEthernet0/0/1] mpls
[AR651-B-GigabitEthernet0/0/1] quit

[AR651-B] static-lsp egress LSP-A-B incoming-interface GigabitEthernet0/0/1
     in-label 300
[AR651-B] return
```

Verificación del LSP estático

```
<HUAWEI> display mpls interface
<HUAWEI> display mpls static-lsp
<HUAWEI> display mpls static-lsp verbose
<HUAWEI> display mpls lsp
```

Desde AR651-A se prueba el LSP mediante el formato indicado por la guía [2, tarea 2.4]:

```
<AR651-A> ping lsp -c 5 ipv4 4.4.4.4 32
```

Si el equipo no admite esa variante, se consulta la ayuda:

```
<AR651-A> ping lsp ?
```

Algunas versiones de VRP utilizan la siguiente sintaxis:

```
<AR651-A> ping lsp ip 4.4.4.4 32
```

Prueba de falla del LSP estático

En AR651-A se apaga la interfaz conectada con S6730H-C, como indica la prueba de falla de la guía [2, tarea 2.5]:

```
<AR651-A> system-view
[AR651-A] interface GigabitEthernet0/0/1
[AR651-A-GigabitEthernet0/0/1] shutdown
[AR651-A-GigabitEthernet0/0/1] quit
[AR651-A] return
```

Después de esperar aproximadamente 30 segundos se comprueba el estado del LSP:

```
<AR651-A> display mpls static-lsp
<AR651-A> ping lsp -c 5 ipv4 4.4.4.4 32
```

Debido a que la topología es lineal y no existe otra ruta, el equipo puede indicar que el LSP especificado no existe. Este resultado confirma que el LSP estático no puede redirigirse automáticamente.

El enlace debe restaurarse antes de continuar:

```
<AR651-A> system-view
[AR651-A] interface GigabitEthernet0/0/1
[AR651-A-GigabitEthernet0/0/1] undo shutdown
[AR651-A-GigabitEthernet0/0/1] quit
[AR651-A] return
```

Después de restaurar la interfaz se verifica nuevamente:

```
<AR651-A> display ospf peer brief
<AR651-A> display mpls static-lsp
<AR651-A> ping lsp -c 5 ipv4 4.4.4.4 32
```

3.1.3. Eliminación del LSP estático

La eliminación de las entradas estáticas antes de habilitar LDP corresponde a la tarea 3.1 de la guía de laboratorio [2, tarea 3.1].

En AR651-A:

```
<AR651-A> system-view
[AR651-A] undo static-lsp ingress LSP-A-B
[AR651-A] return
```

En S6730H-C:

```
<S6730H-C> system-view
```

```
[S6730H-C] undo static-lsp transit LSP-A-B
[S6730H-C] return
```

En S6730H-D:

```
<S6730H-D> system-view
[S6730H-D] undo static-lsp transit LSP-A-B
[S6730H-D] return
```

En AR651-B:

```
<AR651-B> system-view
[AR651-B] undo static-lsp egress LSP-A-B
[AR651-B] return
```

Se confirma que las entradas estáticas fueron eliminadas:

```
<HUAWEI> display mpls static-lsp
```

3.1.4. Configuración de LDP

La configuración global de MPLS y la habilitación de MPLS en las interfaces se conservan. Únicamente se elimina el LSP estático y se activa LDP [2, tareas 3.1–3.2].

Configuración de LDP en AR651-A

```
<AR651-A> system-view
[AR651-A] mpls ldp
[AR651-A-mpls-ldp] quit

[AR651-A] interface GigabitEthernet0/0/1
[AR651-A-GigabitEthernet0/0/1] mpls ldp
[AR651-A-GigabitEthernet0/0/1] quit
[AR651-A] return
```

Configuración de LDP en S6730H-C

```
<S6730H-C> system-view
[S6730H-C] mpls ldp
[S6730H-C-mpls-ldp] quit

[S6730H-C] interface GigabitEthernet0/0/1
[S6730H-C-GigabitEthernet0/0/1] mpls ldp
[S6730H-C-GigabitEthernet0/0/1] quit

[S6730H-C] interface GigabitEthernet0/0/2
[S6730H-C-GigabitEthernet0/0/2] mpls ldp
[S6730H-C-GigabitEthernet0/0/2] quit
[S6730H-C] return
```

Configuración de LDP en S6730H-D

```
<S6730H-D> system-view
[S6730H-D] mpls ldp
[S6730H-D-mpls-ldp] quit

[S6730H-D] interface GigabitEthernet0/0/1
[S6730H-D-GigabitEthernet0/0/1] mpls ldp
[S6730H-D-GigabitEthernet0/0/1] quit

[S6730H-D] interface GigabitEthernet0/0/2
[S6730H-D-GigabitEthernet0/0/2] mpls ldp
[S6730H-D-GigabitEthernet0/0/2] quit
[S6730H-D] return
```

Configuración de LDP en AR651-B

```
<AR651-B> system-view
[AR651-B] mpls ldp
[AR651-B-mpls-ldp] quit

[AR651-B] interface GigabitEthernet0/0/1
[AR651-B-GigabitEthernet0/0/1] mpls ldp
[AR651-B-GigabitEthernet0/0/1] quit
[AR651-B] return
```

Verificación de LDP

Los siguientes comandos permiten verificar las interfaces, vecinos, sesiones y etiquetas de LDP, conforme a las verificaciones de la guía de laboratorio [2, tareas 3.3–3.4]:

```
<HUAWEI> display mpls ldp interface
<HUAWEI> display mpls ldp peer
<HUAWEI> display mpls ldp session
<HUAWEI> display mpls ldp session verbose
<HUAWEI> display mpls ldp lsp
<HUAWEI> display mpls lsp
```

Una sesión LDP completamente establecida debe aparecer en estado `Operational`.

Desde AR651-A se comprueba el plano de datos siguiendo la tarea 3.5 [2, tarea 3.5]:

```
<AR651-A> ping lsp -c 5 ipv4 4.4.4.4 32
<AR651-A> tracert lsp ipv4 4.4.4.4 32
```

Si la versión instalada utiliza otra sintaxis:

```
<AR651-A> ping lsp ?
<AR651-A> tracert lsp ?
```

Las posibles variantes son:

```
<AR651-A> ping lsp ip 4.4.4.4 32
```

```
<AR651-A> tracert lsp ip 4.4.4.4 32
```

Prueba de falla con LDP

En AR651-A se apaga temporalmente la interfaz de salida, siguiendo la prueba de resiliencia de la guía [2, tarea 3.6]:

```
<AR651-A> system-view
[AR651-A] interface GigabitEthernet0/0/1
[AR651-A-GigabitEthernet0/0/1] shutdown
[AR651-A-GigabitEthernet0/0/1] quit
[AR651-A] return
```

Después de esperar se comprueba el LSP:

```
<AR651-A> display mpls ldp session
<AR651-A> display mpls ldp lsp
<AR651-A> ping lsp -c 5 ipv4 4.4.4.4 32
```

En la topología lineal no existe una ruta alternativa. Por esta razón, LDP solamente puede converger por otro camino si se agrega un segundo enlace físico y OSPF aprende una ruta alternativa.

Finalmente se restaura la interfaz:

```
<AR651-A> system-view
[AR651-A] interface GigabitEthernet0/0/1
[AR651-A-GigabitEthernet0/0/1] undo shutdown
[AR651-A-GigabitEthernet0/0/1] quit
[AR651-A] return
```

3.1.5. Cambio del modo de anuncio a DoD

La inspección del modo de anuncio, el cambio a DoD en ambos extremos del enlace y la restauración del modo predeterminado se basan en el experimento de la parte 4 de la guía MPLS [2, parte 4].

Antes de modificar el modo de anuncio, se comprueba la configuración actual en S6730H-C:

```
<S6730H-C> display mpls ldp interface
<S6730H-C> display mpls ldp session verbose
```

El modo predeterminado es DU, correspondiente a *Downstream Unsolicited*. Para comprobar la sintaxis disponible en el equipo:

```
<S6730H-C> system-view
[S6730H-C] interface GigabitEthernet0/0/2
[S6730H-C-GigabitEthernet0/0/2] mpls ldp ?
[S6730H-C-GigabitEthernet0/0/2] quit
[S6730H-C] return
```

Configuración de DoD en S6730H-C

```
<S6730H-C> system-view
[S6730H-C] interface GigabitEthernet0/0/2
[S6730H-C-GigabitEthernet0/0/2] mpls ldp advertisement dod
[S6730H-C-GigabitEthernet0/0/2] quit
[S6730H-C] return
```

Configuración de DoD en S6730H-D

```
<S6730H-D> system-view
[S6730H-D] interface GigabitEthernet0/0/1
[S6730H-D-GigabitEthernet0/0/1] mpls ldp advertisement dod
[S6730H-D-GigabitEthernet0/0/1] quit
[S6730H-D] return
```

El cambio del modo de anuncio puede provocar el reinicio temporal de la sesión LDP. Después del cambio se verifica:

```
<S6730H-C> display mpls ldp session verbose
<S6730H-C> display mpls ldp lsp
```

Restauración del modo DU

En S6730H-C:

```
<S6730H-C> system-view
[S6730H-C] interface GigabitEthernet0/0/2
[S6730H-C-GigabitEthernet0/0/2] mpls ldp advertisement du
[S6730H-C-GigabitEthernet0/0/2] quit
[S6730H-C] return
```

En S6730H-D:

```
<S6730H-D> system-view
[S6730H-D] interface GigabitEthernet0/0/1
[S6730H-D-GigabitEthernet0/0/1] mpls ldp advertisement du
[S6730H-D-GigabitEthernet0/0/1] quit
[S6730H-D] return
```

También puede restaurarse el valor predeterminado con el comando `undo`, si la versión de VRP instalada lo admite:

```
[S6730H-C-GigabitEthernet0/0/2] undo mpls ldp advertisement
[S6730H-D-GigabitEthernet0/0/1] undo mpls ldp advertisement
```

3.1.6. Comandos de diagnóstico

Como recurso complementario para las pruebas de tráfico, la página de descargas de iPerf ofrece los binarios de iPerf3 y las instrucciones de instalación para distintas plataformas [7].

Los siguientes comandos pueden utilizarse en los cuatro dispositivos para identificar problemas de interfaces, direccionamiento, OSPF, MPLS y LDP:

```
<HUAWEI> display interface brief
<HUAWEI> display ip interface brief
<HUAWEI> display ospf peer brief
<HUAWEI> display ip routing-table
<HUAWEI> display mpls interface
<HUAWEI> display mpls static-lsp
<HUAWEI> display mpls static-lsp verbose
<HUAWEI> display mpls ldp interface
<HUAWEI> display mpls ldp peer
<HUAWEI> display mpls ldp session
<HUAWEI> display mpls ldp session verbose
<HUAWEI> display mpls ldp lsp
<HUAWEI> display mpls lsp
<HUAWEI> display current-configuration configuration mpls
```

3.1.7. Almacenamiento de la configuración

La configuración solamente debe guardarse cuando lo autorice el instructor:

```
<HUAWEI> save
```

3.1.8. Limpieza final

Los siguientes comandos eliminan la configuración almacenada y reinician el dispositivo y corresponden a la limpieza final propuesta en la guía [2, apartado Cleanup]. Solamente deben utilizarse cuando lo indique el instructor:

```
<HUAWEI> reset saved-configuration
<HUAWEI> reboot fast
```

En algunas versiones de VRP el segundo comando puede aparecer como `reboot-fast`. La sintaxis disponible puede comprobarse antes de ejecutarlo:

```
<HUAWEI> reboot ?
```

3.1.9. Configuración del acceso SSH

Para permitir la administración remota del router se generó un par de claves RSA, se creó un usuario local con privilegios administrativos y se habilitó el servicio STelnet de Huawei. Este procedimiento se basa en la configuración de acceso SSH de la guía de operación y mantenimiento [5, sec. 1.4.2, paso 3] y en las configuraciones documentadas en el laboratorio 14 [6, pp. 2–7], con el usuario y el direccionamiento adaptados al proyecto.

Primero se generó el par de claves RSA utilizado por el servidor SSH:

```
<AR651-A> system-view
[AR651-A] rsa local-key-pair create
```

Posteriormente, se creó el usuario local `samuel`, se le asignó el nivel máximo de privilegios y se habilitó el servicio SSH:

```
[AR651-A] aaa
[AR651-A-aaa] local-user samuel password cipher hola1234
```

```
[AR651-A-aaa] local-user samuel privilege level 15
[AR651-A-aaa] local-user samuel service-type ssh
[AR651-A-aaa] quit
```

Las líneas VTY se configuraron para utilizar autenticación AAA y aceptar únicamente conexiones mediante SSH:

```
[AR651-A] user-interface vty 0 4
[AR651-A-ui-vty0-4] authentication-mode aaa
[AR651-A-ui-vty0-4] protocol inbound ssh
[AR651-A-ui-vty0-4] quit
```

Finalmente, se permitió el acceso SSH mediante todas las interfaces y se habilitó el servicio STelnet:

```
[AR651-A] ssh server permit interface all
[AR651-A] stelnet server enable
[AR651-A] return
```

La configuración se verificó mediante los siguientes comandos:

```
<AR651-A> display ssh server status
<AR651-A> display ssh user-information
<AR651-A> display current-configuration configuration user-interface
```

Desde una computadora con conectividad hacia el router, el acceso se realiza con el siguiente comando:

```
ssh samuel@10.0.12.1
```

3.2. Generación y verificación de tráfico sobre MPLS

Una vez establecidas las sesiones LDP, se agregaron dos redes de acceso para generar tráfico de extremo a extremo. La primera computadora se conectó al router AR651-A y funcionó como cliente de *iPerf3*; la segunda se conectó al router AR651-B y funcionó como servidor. De esta manera, el tráfico atravesó los cuatro dispositivos del núcleo: AR651-A, S6730H-C, S6730H-D y AR651-B.

La modalidad cliente-servidor y las opciones de las pruebas TCP y UDP se fundamentan en el manual de iPerf3 [8].

3.2.1. Direccionamiento de las redes de prueba

La tabla 3.1 presenta el direccionamiento utilizado. En ambas redes se utilizó una máscara de 30 bits, equivalente a 255 . 255 . 255 . 252.

Cuadro 3.1: Direccionamiento para las pruebas de tráfico

Dispositivo	Interfaz	Dirección IP	Puerta de enlace
Computadora generadora	Ethernet	10.10.10.1/30	10.10.10.2
AR651-A	GE0/0/3	10.10.10.2/30	No aplica
AR651-B	GE0/0/3	20.20.20.2/30	No aplica
Computadora receptora	Ethernet	20.20.20.1/30	20.20.20.2

Las interfaces `GigabitEthernet0/0/3` son interfaces de acceso hacia las computadoras. Por esta razón, no se habilitó MPLS ni LDP directamente en ellas. El etiquetado MPLS se aplicó en las interfaces que pertenecen al núcleo de la red [2, tarea 2.2].

3.2.2. Configuración de la red de acceso en AR651-A

La asignación de direcciones a las interfaces y el anuncio de las redes de acceso siguen el procedimiento de configuración de interfaces y OSPF de Huawei, adaptado a estas subredes [5, sec. 1.4.2, pasos 1–2].

En AR651-A se configuró la dirección `10.10.10.2/30` y se anunció la red mediante OSPF.

```
<AR651-A> system-view
[AR651-A] interface GigabitEthernet0/0/3
[AR651-A-GigabitEthernet0/0/3] ip address 10.10.10.2 255.255.255.252
[AR651-A-GigabitEthernet0/0/3] undo shutdown
[AR651-A-GigabitEthernet0/0/3] quit
[AR651-A] ospf 1
[AR651-A-ospf-1] area 0.0.0.0
[AR651-A-ospf-1-area-0.0.0.0] network 10.10.10.0 0.0.0.3
[AR651-A-ospf-1-area-0.0.0.0] quit
[AR651-A-ospf-1] quit
[AR651-A] return
```

3.2.3. Configuración de la red de acceso en AR651-B

En AR651-B se configuró la dirección `20.20.20.2/30` y también se anunció la red mediante OSPF.

```
<AR651-B> system-view
[AR651-B] interface GigabitEthernet0/0/3
[AR651-B-GigabitEthernet0/0/3] ip address 20.20.20.2 255.255.255.252
[AR651-B-GigabitEthernet0/0/3] undo shutdown
[AR651-B-GigabitEthernet0/0/3] quit
[AR651-B] ospf 1
[AR651-B-ospf-1] area 0.0.0.0
[AR651-B-ospf-1-area-0.0.0.0] network 20.20.20.0 0.0.0.3
[AR651-B-ospf-1-area-0.0.0.0] quit
[AR651-B-ospf-1] quit
[AR651-B] return
```

La propagación de las dos redes se verificó con los siguientes comandos:

```
<AR651-A> display ip routing-table 20.20.20.0
```

```
<AR651-B> display ip routing-table 10.10.10.0
<AR651-A> ping -a 10.10.10.2 20.20.20.2
<AR651-B> ping -a 20.20.20.2 10.10.10.2
```

El ping originado en AR651-A obtuvo respuesta de AR651-B con una pérdida del 0 %, lo cual confirmó que OSPF proporcionaba conectividad y una ruta de retorno entre ambos extremos.

3.2.4. Configuración de las computadoras

La asignación de la dirección IPv4, el prefijo y la puerta de enlace mediante `New-NetIPAddress` se basa en la referencia del comando de Microsoft [9].

En la computadora generadora se configuraron los siguientes parámetros:

```
Direccion IPv4: 10.10.10.1
Longitud de prefijo: 30
Puerta de enlace: 10.10.10.2
```

La configuración puede aplicarse desde PowerShell con privilegios de administrador. El índice 13 corresponde a la interfaz Ethernet utilizada en la prueba y debe sustituirse si Windows muestra un valor diferente.

```
PS> New-NetIPAddress -InterfaceIndex 13 ‘
      -IPAddress 10.10.10.1 ‘
      -PrefixLength 30 ‘
      -DefaultGateway 10.10.10.2
```

En la computadora receptora se configuraron los siguientes valores:

```
Direccion IPv4: 20.20.20.1
Longitud de prefijo: 30
Puerta de enlace: 20.20.20.2
```

```
PS> New-NetIPAddress -InterfaceAlias "Ethernet" ‘
      -IPAddress 20.20.20.1 ‘
      -PrefixLength 30 ‘
      -DefaultGateway 20.20.20.2
```

Durante la implementación se detectó que Windows seleccionaba la interfaz virtual de ZeroTier para alcanzar la red `20.20.20.0/30`. La interfaz elegida por el sistema se comprobó con el siguiente comando: La función de `Find-NetRoute` para identificar la dirección local y la ruta hacia un destino está documentada por Microsoft [10].

```
PS> Find-NetRoute -RemoteIPAddress 20.20.20.2
```

Para garantizar que el tráfico de laboratorio utilizara la interfaz Ethernet y el router AR651-A, se agregó una ruta más específica desde una consola de PowerShell ejecutada como administrador: La sintaxis para establecer el prefijo de destino, la interfaz, el siguiente salto y la métrica corresponde a `New-NetRoute` [11].

```
PS> New-NetRoute -InterfaceIndex 13 ‘
      -DestinationPrefix "20.20.20.0/30" ‘
      -NextHop 10.10.10.2 ‘
      -RouteMetric 1
```

Después del cambio, `Find-NetRoute` mostró la interfaz Ethernet, el `InterfaceIndex` `13` y el siguiente salto `10.10.10.2`. La conectividad se validó con:

```
PS> ping 10.10.10.2
PS> ping 20.20.20.2
PS> ping 20.20.20.1
PS> tracert -d 20.20.20.1
```

3.2.5. Creación de LSP para las redes de acceso

Inicialmente, la salida de `display mpls ldp lsp` mostraba LSP únicamente para las direcciones de loopback `/32`. En ese estado, LDP estaba operativo, pero el tráfico entre las computadoras se reenviaba como IPv4 convencional debido a que no existía una FEC para las redes de acceso.

Para esta topología de laboratorio se configuró `lsp-trigger all` en los cuatro dispositivos. Con ello, las rutas estáticas y las rutas aprendidas por el IGP pueden activar la creación de LSP [12]. Este comando debe utilizarse con precaución en redes de producción porque puede aumentar la cantidad de etiquetas utilizadas [13].

Configuración en AR651-A

```
<AR651-A> system-view
[AR651-A] mpls
[AR651-A-mpls] lsp-trigger all
[AR651-A-mpls] quit
[AR651-A] return
```

Configuración en S6730H-C

```
<S6730H-C> system-view
[S6730H-C] mpls
[S6730H-C-mpls] lsp-trigger all
[S6730H-C-mpls] quit
[S6730H-C] return
```

Configuración en S6730H-D

```
<S6730H-D> system-view
[S6730H-D] mpls
[S6730H-D-mpls] lsp-trigger all
[S6730H-D-mpls] quit
[S6730H-D] return
```

Configuración en AR651-B

```
<AR651-B> system-view
[AR651-B] mpls
```

```
[AR651-B-mpls] lsp-trigger all
[AR651-B-mpls] quit
[AR651-B] return
```

En versiones de VRP que no presentan el comando dentro de la vista MPLS, se debe consultar la familia IPv4 de LDP y utilizar solamente la variante admitida por el dispositivo: La variante de `lsp-trigger` en la familia IPv4 de LDP se documenta en la referencia de comandos de Huawei [13].

```
[HUAWEI] mpls ldp
[HUAWEI-mpls-ldp] ipv4-family
[HUAWEI-mpls-ldp-ipv4] lsp-trigger ?
[HUAWEI-mpls-ldp-ipv4] lsp-trigger all
```

3.2.6. Verificación del etiquetado MPLS

OSPF y MPLS cumplen funciones diferentes. OSPF calcula la ruta IP hacia el destino, mientras que LDP distribuye las etiquetas que permiten reenviar los paquetes a través de un LSP [2, partes 1 y 3]. Por lo tanto, la presencia de vecinos OSPF y sesiones LDP no era suficiente para demostrar que el tráfico de las computadoras estuviera etiquetado.

La verificación se realizó con: Los comandos de consulta de vecinos, sesiones y LSP corresponden a las verificaciones de la guía de laboratorio [2, tareas 3.3–3.4].

```
<HUAWEI> display mpls ldp peer
<HUAWEI> display mpls ldp session
<HUAWEI> display mpls ldp lsp
<HUAWEI> display mpls lsp
```

En AR651-A se obtuvo una entrada para la FEC `20.20.20.0/30`:

```
DestAddress/Mask   In/OutLabel   NextHop      OutInterface
20.20.20.0/30      NULL/1029     10.0.12.2    GE0/0/1
```

Esta entrada confirma que AR651-A funciona como LER de ingreso para la red `20.20.20.0/30`: recibe un paquete IPv4 sin etiqueta y le inserta la etiqueta 1029 antes de enviarlo por `GE0/0/1`. En los switches de tránsito se realiza la sustitución de etiquetas y, cerca del extremo de salida, puede observarse PHP mediante la etiqueta *Implicit NULL* [2, tarea 2.3 y pregunta 5.2].

Para comprobar el sentido inverso, en AR651-B se debe verificar una entrada con etiqueta de salida para la FEC `10.10.10.0/30`:

```
<AR651-B> display mpls ldp lsp
<AR651-B> display mpls lsp
```

3.2.7. Instalación y preparación de iPerf3

La página de iPerf reúne las opciones de descarga para Windows [7]. La instalación por identificador y versión, con coincidencia exacta, utiliza las opciones de `winget install` documentadas por Microsoft [14].

En las computadoras con Windows se instaló iPerf3 versión 3.20 mediante `winget`:

```
PS> winget install --id ar51an.iPerf3 --version 3.20 --exact
```

Después de la instalación fue necesario cerrar y abrir PowerShell para que la modificación de la variable `PATH` entrara en vigor. La versión se comprobó con:

```
PS> iperf3 --version
```

En la computadora receptora se permitieron los puertos TCP y UDP 5201 desde PowerShell con privilegios de administrador: El puerto predeterminado de iPerf3 es 5201 [8]; las reglas de entrada se crean mediante el comando `New-NetFirewallRule` [15].

```
PS> New-NetFirewallRule -DisplayName "iPerf3 TCP 5201" ‘
      -Direction Inbound -Protocol TCP -LocalPort 5201 -Action Allow
PS> New-NetFirewallRule -DisplayName "iPerf3 UDP 5201" ‘
      -Direction Inbound -Protocol UDP -LocalPort 5201 -Action Allow
```

3.2.8. Pruebas de rendimiento con TCP

Las opciones `-s`, `-c`, `-t` e `-i` seleccionan el modo servidor, el destino del cliente, la duración y el intervalo de reporte, respectivamente [8].

En la computadora receptora, con dirección `20.20.20.1`, se inició el servidor:

```
PS> iperf3 -s
```

En la computadora generadora, con dirección `10.10.10.1`, se ejecutó una prueba TCP de 120 segundos:

```
PS> iperf3 -c 20.20.20.1 -t 120 -i 1
```

Durante la prueba se observó un rendimiento estable cercano a `946-948 Mbit/s`. Este resultado es coherente con la capacidad efectiva de un enlace Gigabit Ethernet, debido a que parte del ancho de banda se utiliza en las cabeceras Ethernet, MPLS, IP y TCP.

La prueba TCP predeterminada es unidireccional: el cliente envía los datos y el servidor devuelve confirmaciones TCP. Estas confirmaciones no representan un flujo de datos equivalente en sentido contrario. Para probar el sentido inverso se utiliza la opción `-R` [8]:

```
PS> iperf3 -c 20.20.20.1 -t 120 -i 1 -R
```

Para cuatro flujos TCP simultáneos se puede utilizar [8]:

```
PS> iperf3 -c 20.20.20.1 -t 120 -i 1 -P 4
```

3.2.9. Pruebas de rendimiento con UDP

A diferencia de TCP, UDP no ajusta automáticamente la velocidad ni retransmite datagramas perdidos. La opción `-b` especifica el ancho de banda solicitado [8]. La prueba debe comenzar con una tasa moderada y aumentar de forma gradual.

Prueba UDP a 500 Mbit/s:

```
PS> iperf3 -c 20.20.20.1 -u -b 500M -t 120 -i 1
```

Prueba UDP a 900 Mbit/s:

```
PS> iperf3 -c 20.20.20.1 -u -b 900M -t 120 -i 1
```

Prueba UDP en sentido inverso:

```
PS> iperf3 -c 20.20.20.1 -u -b 900M -t 120 -i 1 -R
```

Prueba UDP bidireccional a 400 Mbit/s por sentido [8]:

```
PS> iperf3 -c 20.20.20.1 -u -b 400M -t 120 -i 1 --bidir
```

En los resultados UDP deben analizarse el caudal recibido, el *jitter*, la cantidad de datagramas perdidos y el porcentaje de pérdida. Una tasa solicitada cercana al límite físico del enlace puede producir pérdidas aunque la red se encuentre operativa.

3.2.10. Criterios de aceptación de la prueba

La prueba se considera satisfactoria cuando se cumplen los siguientes criterios:

- Las adyacencias OSPF se encuentran en estado `Full`.
- Las sesiones LDP se encuentran en estado `Operational`.
- AR651-A aprende la red `20.20.20.0/30` y AR651-B aprende la red `10.10.10.0/30`.
- Existe conectividad entre `10.10.10.1` y `20.20.20.1`.
- AR651-A muestra una etiqueta de salida para la FEC `20.20.20.0/30`.
- AR651-B muestra una etiqueta de salida para la FEC `10.10.10.0/30`.
- iPerf3 completa las pruebas TCP y UDP sin errores de conexión.
- Los resultados UDP registran el caudal, el *jitter* y el porcentaje de datagramas perdidos.

Con estas pruebas se demostró no solamente la conectividad IP proporcionada por OSPF, sino también la existencia de LSP dinámicos para las redes de acceso y el transporte efectivo de tráfico generado con iPerf3 sobre el núcleo MPLS.

Capítulo 4

Configuración y automatización con Netdata

4.1. Creación de Dashboards en NetData

Para la creación de dashboards en NetData, se pueden seguir los siguientes pasos:

- Acceder a la interfaz web de NetData a través del navegador, utilizando la dirección IP del servidor donde está instalado NetData y el puerto correspondiente (por defecto es el puerto 19999). O ingresar a NetData cloud.
- Navegar a la sección de Dashboards o Tableros en la interfaz de NetData.
- Seleccionar la opción deCrear nuevo dashboard o Agregar tablero para iniciar el proceso de creación. [16]

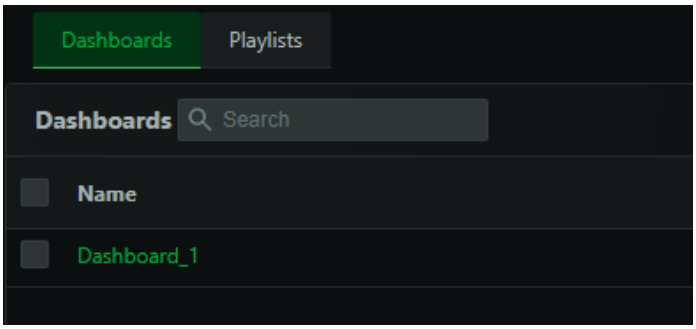


Figura 4.1: Formación de Dashboard en NetData

En este caso se le colocó de nombre: Dashboard_1

- Cuando se ingresa por prmera vez, se puede observar que NetData recomienda ciertas plantillas de dashboards predefinidas, las cuales se pueden utilizar como base para crear un nuevo dashboard personalizado. Se puede seleccionar una plantilla existente o crear un dashboard desde cero según las necesidades del usuario. En este caso, se utiliza la de NOC.

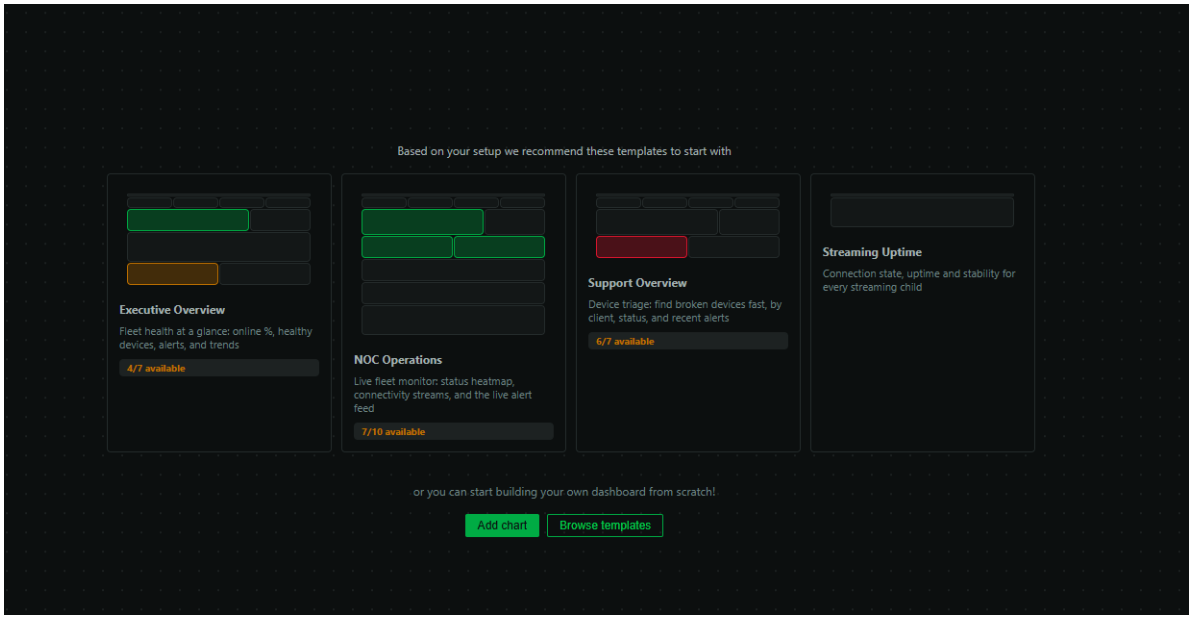


Figura 4.2: Primer ingreso al Dashboard de NetData

- Se seleccionan los dispositivos que se desean monitorear en el dashboard, como servidores, routers, switches u otros dispositivos de red. Se pueden agregar múltiples dispositivos al mismo dashboard para tener una visión completa del estado de la red.

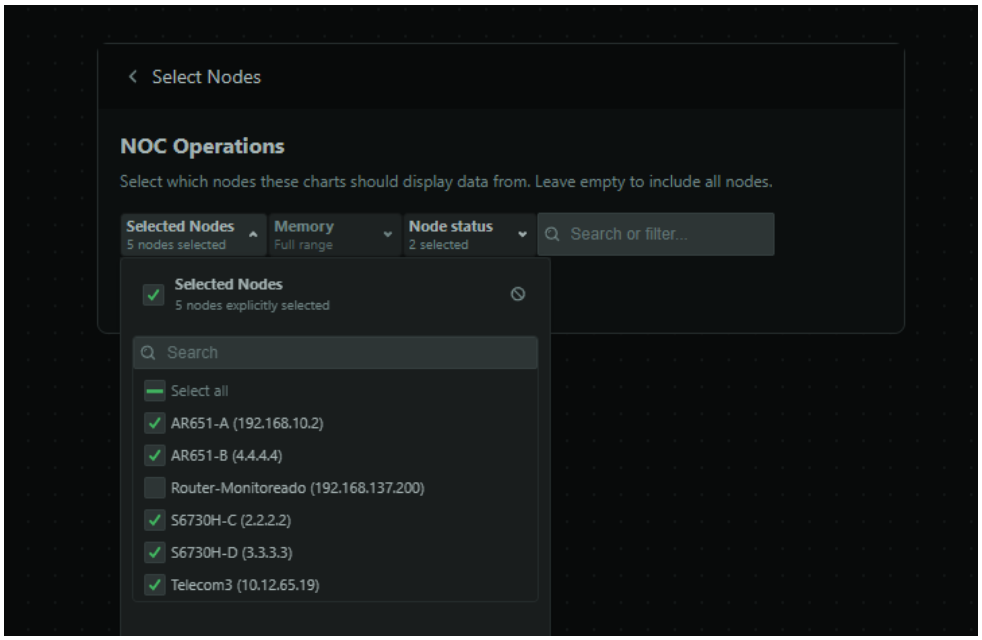


Figura 4.3: Dispositivos seleccionados para el Dashboard de NetData

- Luego de guardar los cambios, ya se puede acceder al dashboard creado y visualizar los datos en tiempo real. Se pueden personalizar los gráficos, agregar widgets adicionales y configurar alertas según las necesidades del usuario.

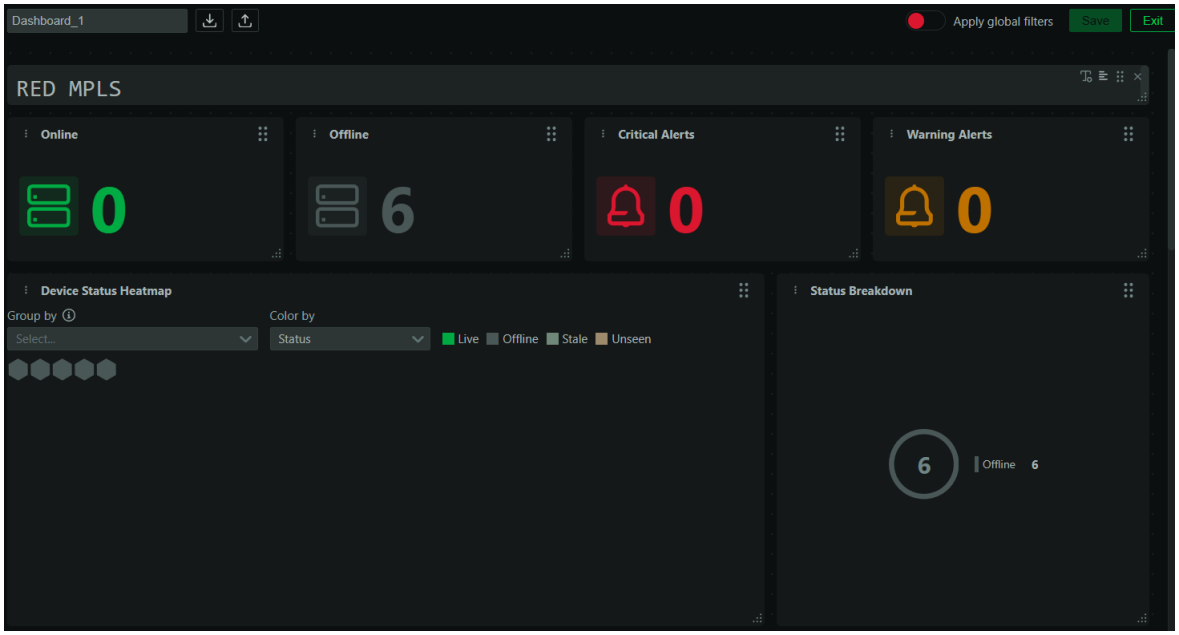


Figura 4.4: Dashboard creado en NetData

4.2. Editar Dashboards en NetData

Para editar el tablero correspondiente, se debe de ingresar al dashboard y dar clic en el botón de edit. Luego de esto, se puede observar que aparece una barra con opciones a la izquierda, ahi se puede colocar diferentes bloques de funcionamiento.

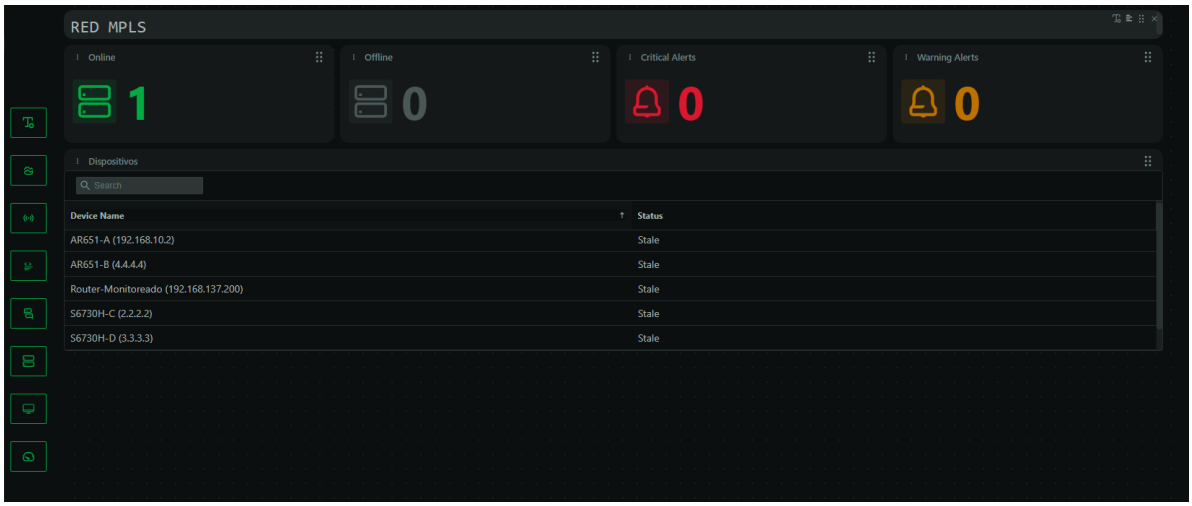


Figura 4.5: Opciones de bloques para colocar en el Dashboard

- En este caso, se desean colocar ciertos datos de los dispositivos, como tráfico, carga de CPU, entre otros, para esto se ingresa al bloque de métricas, se selecciona el dispositivo deseado y se selecciona de las categorías y subcategorías que métrica colocar.

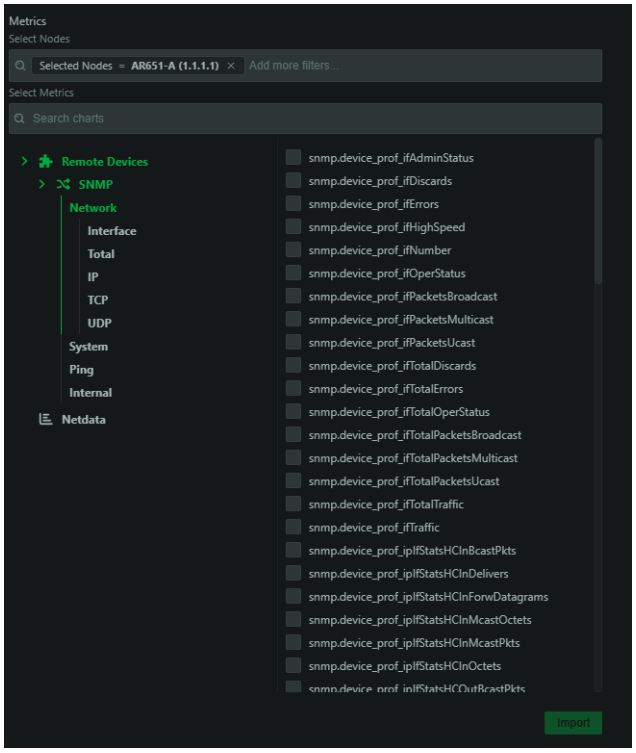


Figura 4.6: Colocación de datos en el dashboard

- En este caso, se colocaron gráficas de los puertos utilizados en cada router. También se colocó un bloque de texto, el cual permite elegir si es título, subtitulo, o parrafo, en este caso se seleccionó título.



Figura 4.7: Gráfica puertos, ejemplo de router A

- Cuando son bloques de gráficas de métricas las que se colocan, en la parte superior del bloque (dashboard en modo edición), aparecen opciones para poder filtrar los datos, como por ejemplo, colocar el valor en máximo, mínimo, promedio, entre otros.

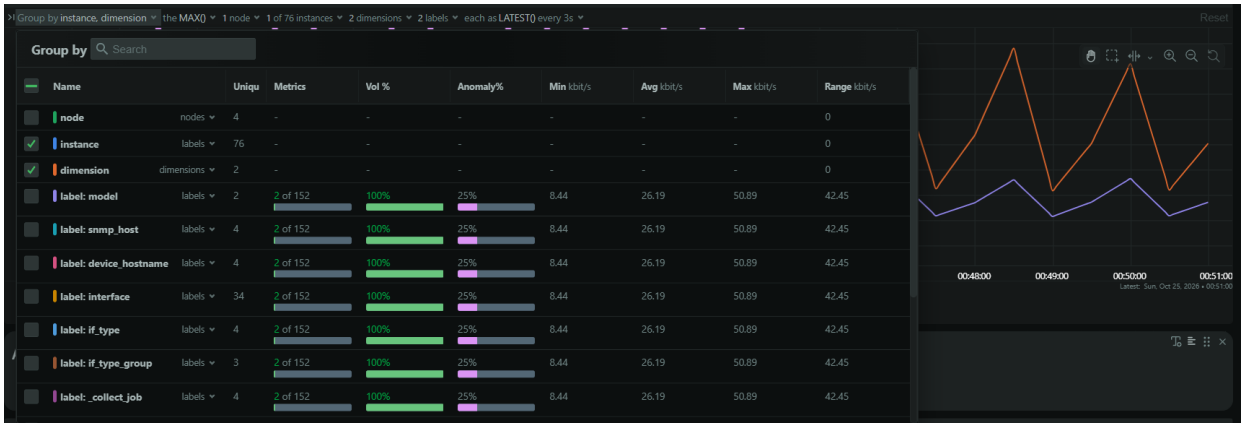


Figura 4.8: Ejemplo de filtro colocado en las gráficas de interfaces

4.3. Configuración de una alerta de latencia para los dispositivos SNMP

Una vez comprobada la recolección de métricas de los cuatro dispositivos Huawei, se configuró una alerta personalizada para supervisar la latencia de comunicación. Netdata obtiene esta información mediante las pruebas ICMP realizadas por el recolector SNMP.

Los cuatro dispositivos utilizan el contexto de métricas:

```
snmp.device_ping_rtt
```

Este contexto contiene las dimensiones `min`, `max` y `avg`, correspondientes a la latencia mínima, máxima y promedio. Para la alerta se utilizó la dimensión `avg`.

Primero, se creó el directorio destinado a las configuraciones personalizadas de salud, en caso de que este no existiera:

```
sudo mkdir -p /etc/netdata/health.d
```

Posteriormente, se creó el archivo de configuración de alertas:

```
sudo nano /etc/netdata/health.d/snmp_custom.conf
```

Dentro del archivo se agregó la siguiente plantilla:

```
# Alerta personalizada de latencia SNMP

template: snmp_high_latency
      on: snmp.device_ping_rtt
   class: Latency
    type: Network
component: SNMP
   lookup: average -1m of avg
    every: 30s
    units: ms
     warn: $this > 100
     crit: $this > 250
  summary: Latencia elevada hacia ${host}
     info: La latencia promedio del dispositivo supero los limites
    configurados.
       to: sysadmin
```

La alerta calcula el promedio de la dimensión `avg` durante el último minuto y evalúa la condición cada 30 segundos. Los límites configurados fueron los siguientes:

- Estado normal: latencia igual o menor que 100 ms.
- Advertencia (`WARNING`): latencia mayor que 100 ms.
- Estado crítico (`CRITICAL`): latencia mayor que 250 ms.

Después de guardar el archivo, se recargó el sistema de alertas de Netdata sin reiniciar completamente el servicio:

```
sudo netdatacli reload-health
```

Para comprobar que la plantilla fue aplicada a los cuatro dispositivos, se ejecutó la siguiente consulta:

```
for nodo in AR651-A S6730H-C S6730H-D AR651-B; do
  echo "===== $nodo ====="
  curl -s
  "http://127.0.0.1:19999/host/$nodo/api/v1/alarms?all"
  | grep -A8 -B2 'snmp_high_latency'
done
```

La plantilla generó automáticamente una instancia de la alerta para cada dispositivo debido a que todos comparten el contexto `snmp.device_ping_rtt`. Mientras la latencia permanece dentro de los límites establecidos, la alerta presenta el estado `CLEAR`. [17]

Finalmente, se realizó una prueba controlada modificando temporalmente los límites:

```
warn: $this > 1
crit: $this > 5
```

Después de recargar nuevamente las alertas, Netdata detectó que la latencia superaba el límite crítico y mostró el estado `CRITICAL` en Netdata Cloud. Esto permitió comprobar que la plantilla funcionaba correctamente. Al finalizar la prueba, se restauraron los valores de 100 ms y 250 ms y se aplicó nuevamente la configuración:

```
sudo netdatacli reload-health
```

De esta forma, se implementó una única plantilla capaz de supervisar automáticamente la latencia de los cuatro dispositivos Huawei, evitando crear una alerta independiente para cada equipo.

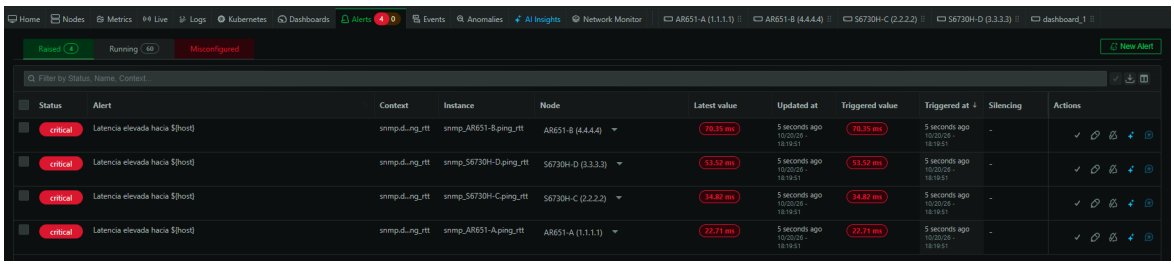


Figura 4.9: Alertas generadas de ICMP elevado.

4.4. Configuración de alertas por fallo en la recolección SNMP

Además de la supervisión de latencia, se implementaron alertas para detectar cuando Netdata deja de recibir métricas SNMP de cualquiera de los cuatro dispositivos Huawei. Esta condición puede producirse por el apagado de un equipo, una interrupción en la red, credenciales SNMP incorrectas o una falla del servicio SNMP. [18]

Netdata genera una gráfica de estado para cada trabajo configurado en el archivo `snmp.conf`. Para identificar estas gráficas se ejecutó el siguiente script:

```
python3 - <<'PY'
import json
import urllib.request

url = "http://127.0.0.1:19999/api/v1/charts"

with urllib.request.urlopen(url) as response:
    charts = json.load(response).get("charts", {})

for chart in charts.values():
    text = " ".join([
        chart.get("id", ""),
        chart.get("context", ""),
        chart.get("title", "")
    ]).lower()

    if "snmp" in text and "collection_status" in text:
        print(
            "ID:", chart.get("id"),
            "| CONTEXT:", chart.get("context"),
            "| DIMENSIONS:",
            list(chart.get("dimensions", {}).keys())
        )
PY
```

La consulta permitió identificar las siguientes gráficas:

- `netdata.go_d_snmp_AR651-A_data_collection_status`
- `netdata.go_d_snmp_S6730H-C_data_collection_status`
- `netdata.go_d_snmp_S6730H-D_data_collection_status`
- `netdata.go_d_snmp_AR651-B_data_collection_status`

Todas las gráficas utilizan el contexto `netdata.plugin_data_collection_status` y contienen las dimensiones `success` y `failed`. Un valor igual a cero en la dimensión `failed` indica que la consulta se realizó correctamente, mientras que un valor mayor que cero indica una falla.

Debido a que este contexto también es utilizado por otros recolectores de Netdata, se crearon alarmas específicas para cada trabajo SNMP. De esta manera se evita que la alerta se aplique accidentalmente a otros plugins.

Se abrió el archivo de alertas personalizadas:

```
sudo nano /etc/netdata/health.d/snmp_custom.conf
```

A continuación, se agregaron las alarmas correspondientes a los cuatro dispositivos:

```
# ============================================================
# Fallo de recoleccion SNMP
# ============================================================

alarm: snmp_collection_failure_AR651_A
   on: netdata.go_d_snmp_AR651-A_data_collection_status
class: Errors
 type: Network
component: SNMP
lookup: average -1m of failed
every: 30s
units: status
 warn: $this > 0
 crit: $this >= 0.8
summary: Fallo de recoleccion SNMP en AR651-A
 info: Netdata no recibe correctamente las metricas SNMP de AR651-A.
   to: sysadmin


alarm: snmp_collection_failure_S6730H_C
   on: netdata.go_d_snmp_S6730H-C_data_collection_status
class: Errors
 type: Network
component: SNMP
lookup: average -1m of failed
every: 30s
units: status
 warn: $this > 0
 crit: $this >= 0.8
summary: Fallo de recoleccion SNMP en S6730H-C
 info: Netdata no recibe correctamente las metricas SNMP de S6730H-C.
   to: sysadmin


alarm: snmp_collection_failure_S6730H_D
   on: netdata.go_d_snmp_S6730H-D_data_collection_status
class: Errors
 type: Network
component: SNMP
lookup: average -1m of failed
every: 30s
units: status
 warn: $this > 0
 crit: $this >= 0.8
summary: Fallo de recoleccion SNMP en S6730H-D
 info: Netdata no recibe correctamente las metricas SNMP de S6730H-D.
   to: sysadmin


alarm: snmp_collection_failure_AR651_B
   on: netdata.go_d_snmp_AR651-B_data_collection_status
class: Errors
 type: Network
component: SNMP
lookup: average -1m of failed
every: 30s
units: status
```

```
 warn: $this > 0
 crit: $this >= 0.8
summary: Fallo de recoleccion SNMP en AR651-B
 info: Netdata no recibe correctamente las metricas SNMP de AR651-B.
   to: sysadmin
```

Cada alarma calcula el promedio de la dimensión `failed` durante el último minuto y evalúa su condición cada 30 segundos. La interpretación de los estados es la siguiente:

- `CLEAR`: la recolección SNMP funciona correctamente.
- `WARNING`: se detectó al menos un fallo durante el último minuto.
- `CRITICAL`: la recolección falló de manera continua durante la mayor parte del último minuto.

Después de guardar el archivo, se recargó el sistema de alertas sin reiniciar completamente el agente:

```
sudo netdatacli reload-health
```

La existencia de las alarmas se comprobó mediante la API local de Netdata:

```
curl -s
'http://127.0.0.1:19999/api/v1/alarms?all'
| grep -A12 -B2 'snmp_collection_failure'
```

También se agregó alerta por pérdidas de ICMP:

```
sudo nano /etc/netdata/go.d/ping.conf
```

```
update_every: 10
autodetection_retry: 0

jobs:
  - name: dispositivos_huawei
    privileged: yes
    packets: 3
    hosts:
      - AR651-A
      - S6730H-C
      - S6730H-D
      - AR651-B
```

4.4.1. Prueba de funcionamiento

Para verificar el funcionamiento de las alarmas se apagó el router `AR651-B` dentro del entorno de simulación. Como consecuencia, la Raspberry Pi dejó de recibir respuestas SNMP desde la dirección `4.4.4.4`.

Inicialmente, Netdata generó una advertencia debido a los primeros intentos fallidos de recolección. Al mantenerse el equipo apagado y continuar los fallos durante el periodo de evaluación, la alarma personalizada cambió al estado `CRITICAL`.

Además de la alarma crítica de recolección SNMP, Netdata presentó una advertencia relacionada con la pérdida de comunicación con el dispositivo. Esto permitió observar que las alertas pueden representar diferentes niveles o causas de una misma incidencia.

La prueba confirmó que el sistema puede detectar la pérdida de conectividad con un dispositivo aun cuando la Raspberry Pi y el agente de Netdata continúan funcionando correctamente. Al encender nuevamente el router y restablecerse las consultas SNMP, la alarma regresó automáticamente al estado `CLEAR`.

Con esta configuración, el sistema supervisa individualmente los cuatro dispositivos y permite identificar cuál equipo dejó de responder, facilitando la detección y localización de fallas dentro de la red MPLS.

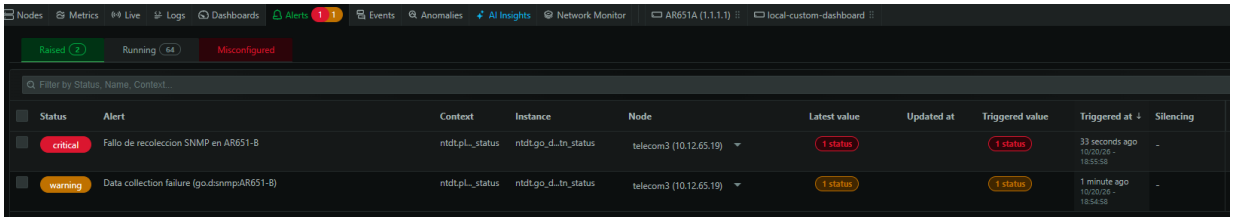


Figura 4.10: Alerta generada por pérdida de comunicación con SNMP.

4.5. Configuración de la topología SNMP en Netdata

Además de recopilar las métricas individuales de los routers y switches, se configuró el módulo de topología SNMP de Netdata. Este módulo permite representar gráficamente los dispositivos administrados y los enlaces existentes entre ellos.

La identificación de los enlaces se realizó mediante LLDP (*Link Layer Discovery Protocol*). Los dispositivos Huawei tenían habilitado este protocolo, por lo que fue posible consultar mediante SNMP la información de sus interfaces y vecinos. [19]

4.5.1. Activación del recolector de topología

Se creó el archivo de configuración del recolector de topología:

```
sudo nano /etc/netdata/go.d/snmp_topology.conf
```

Dentro del archivo se agregó la siguiente configuración:

```
jobs:
  - name: snmp_topology
    update_every: 30
    refresh_every: 1m
```

El parámetro `update_every` establece el intervalo de actualización del recolector, mientras que `refresh_every` indica cada cuánto tiempo Netdata debe volver a descubrir los dispositivos y enlaces de la topología.

4.5.2. Creación del directorio de perfiles personalizados

Los perfiles instalados por Netdata se encuentran en el directorio `/usr/lib/netdata`. Estos archivos no se modificaron directamente, debido a que podrían ser reemplazados durante una actualización del programa.

En su lugar, se creó el directorio destinado a los perfiles personalizados:

```
sudo mkdir -p /etc/netdata/go.d/snmp.profiles
```

4.5.3. Creación del perfil genérico personalizado

Los dispositivos Huawei coincidían tanto con el perfil específico de Huawei como con el perfil genérico de Netdata. Para evitar que ambos perfiles procesaran nuevamente la misma información básica del dispositivo, se creó una versión personalizada del perfil genérico.

El archivo se creó mediante:

```
sudo nano /etc/netdata/go.d/snmp.profiles/generic-device.yaml
```

Se agregó el siguiente contenido:

```
extends:
  - _std-if-mib.yaml
  - _std-ip-mib.yaml
  - _std-tcp-mib.yaml
  - _std-udp-mib.yaml
  - _std-ospf-mib.yaml
  - _std-host-resources-mib.yaml
  - _std-topology-ip-mib.yaml

selector:
  - sysobjectid:
      include:
        - 1.3.6.1.4.*
```

Este perfil conserva la recopilación genérica de interfaces, tráfico, protocolos IP, OSPF y topología de capa 3. La información básica de identificación continúa siendo proporcionada por el perfil específico de Huawei.

4.5.4. Creación del perfil de topología para Huawei

Se creó un segundo perfil para incorporar las tablas de interfaces, direcciones IP y vecinos LLDP de los dispositivos Huawei:

```
sudo nano /etc/netdata/go.d/snmp.profiles/huawei-topology.yaml
```

El contenido utilizado fue el siguiente:

```
extends:
  - _std-topology-interface-mib.yaml
  - _std-topology-ip-mib.yaml
  - _std-topology-lldp-mib.yaml

selector:
  - sysobjectid:
      include:
        - 1.3.6.1.4.1.2011.*
```

El identificador empresarial SNMP de Huawei es `1.3.6.1.4.1.2011`. Por esta razón, el selector aplica el perfil a los routers y switches Huawei detectados por Netdata.

Los perfiles heredados cumplen las siguientes funciones:

- `_std-topology-interface-mib.yaml`: obtiene la información de las interfaces.
- `_std-topology-ip-mib.yaml`: relaciona las direcciones IP con las interfaces.

- `_std-topology-lldp-mib.yaml`: obtiene los vecinos y enlaces descubiertos mediante LLDP.

4.5.5. Asignación de propietarios y permisos

El directorio y los perfiles personalizados se asignaron al usuario `root` y al grupo `netdata`:

```
sudo chown -R root:netdata \
/etc/netdata/go.d/snmp.profiles

sudo chmod 750 \
/etc/netdata/go.d/snmp.profiles
```

Posteriormente, se establecieron permisos de lectura para el servicio de Netdata:

```
sudo chmod 640 \
/etc/netdata/go.d/snmp.profiles/generic-device.yaml \
/etc/netdata/go.d/snmp.profiles/huawei-topology.yaml
```

El permiso `640` permite que el propietario pueda leer y modificar los archivos, mientras que el grupo `netdata` únicamente puede leerlos.

4.5.6. Resolución local de los nombres de los dispositivos

La dirección `1.1.1.1`, utilizada como dirección de administración del router AR651-A, corresponde también a una dirección pública real. Su resolución DNS inversa devuelve el nombre `one.one.one.one`. Como consecuencia, Netdata utilizaba dicho nombre para identificar el router en la topología.

Para establecer localmente los nombres correctos de todos los dispositivos, se modificó el archivo `/etc/hosts`:

```
sudo nano /etc/hosts
```

Se agregaron las siguientes entradas:

```
1.1.1.1    AR651-A
2.2.2.2    S6730H-C
3.3.3.3    S6730H-D
4.4.4.4    AR651-B
```

Estas entradas permiten que la Raspberry Pi asocie cada dirección de administración con el nombre correspondiente del dispositivo. La resolución local tiene prioridad sobre la consulta DNS externa, evitando que el router AR651-A aparezca como `one.one.one.one`.

La resolución del nombre se comprobó con:

```
getent hosts 1.1.1.1
```

El resultado esperado fue:

```
1.1.1.1    AR651-A
```

4.5.7. Aplicación y verificación de la configuración

Después de crear los perfiles y configurar la resolución local de nombres, se reinició el servicio de Netdata:

```
sudo systemctl restart netdata
```

El funcionamiento del recolector de topología se verificó mediante el registro del servicio:

```
sudo journalctl --namespace netdata \
--since "5 minutes ago" --no-pager |
grep -i 'snmp_topology'
```

Finalmente, se accedió en la interfaz de Netdata a la sección *Topology*, seleccionando posteriormente la topología SNMP.

Como resultado, Netdata mostró gráficamente los cuatro dispositivos Huawei y los enlaces existentes entre ellos. La representación corresponde con la topología implementada en el laboratorio y utiliza la información de vecinos LLDP obtenida mediante SNMPv3.

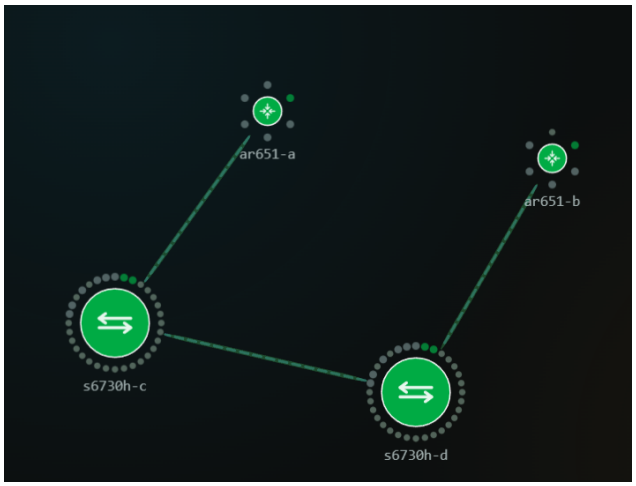


Figura 4.11: Topología SNMP

Al dar clic sobre cualquier dispositivo, se puede observar que se muestran ciertos datos, como por ejemplo sus interfaces, si estan activas o no, vecinos, ciertas etiquetas, etc.



Figura 4.12: Información sobre un dispositivo

NOTA: Esta topología es cambiante, si por ejemplo un dispositivo deja de funcionar, o se cae una interfaz, ese dispositivo deja de estar visible en la red.

4.6. Implementación de una topología de red personalizada

La topología SNMP incorporada en Netdata se genera dinámicamente a partir de información obtenida mediante protocolos y tablas como LLDP, ARP, OSPF y MPLS. Debido a este comportamiento, cuando un dispositivo deja de responder o una adyacencia desaparece de las tablas SNMP, Netdata puede retirar temporalmente el dispositivo o enlace de la representación gráfica.

Por esta razón, se desarrolló una página web personalizada que utiliza Netdata como fuente de información. Netdata continúa realizando la recopilación, almacenamiento y procesamiento de las métricas, mientras que la página se encarga únicamente de representar gráficamente la topología.

4.6.1. Arquitectura de la solución

La solución implementada está compuesta por los siguientes elementos:

- Netdata recopila las métricas SNMP de los cuatro dispositivos Huawei.
- El colector personalizado de OSPF y MPLS LDP publica el estado permanente de cada enlace.
- Un servidor web desarrollado en Python publica la página en el puerto TCP 8080.
- El servidor Python funciona como intermediario entre el navegador y la API local de Netdata.
- Una página HTML representa permanentemente los cuatro dispositivos y los tres enlaces de la red.
- JavaScript consulta periódicamente las métricas de Netdata y actualiza los colores, estados y valores de tráfico.

De esta manera, el navegador no se conecta directamente al puerto 19999 de Netdata. En su lugar, realiza las consultas mediante el servidor de la topología, el cual reenvía las solicitudes hacia Netdata de forma local.

4.6.2. Contenido

Cuadro 4.1: Archivos de la topología personalizada

Archivo	Función
`index.html`	Contiene la interfaz gráfica, el diseño de la red, los dispositivos, enlaces, colores, indicadores de estado y el código JavaScript que consulta las métricas de Netdata.
`server.py`	Implementa un servidor HTTP en Python. Publica la página web en el puerto 8080 y reenvía las consultas hacia la API local de Netdata.
`netdata-topology.service`	Define un servicio de systemd para iniciar automáticamente la aplicación al encender la Raspberry Pi.
`README.txt`	Contiene instrucciones básicas para instalar y ejecutar la aplicación.

4.6.3. Topología representada

La página mantiene una topología fija con los siguientes dispositivos:

```
AR651-A --- S6730H-C --- S6730H-D --- AR651-B
```

Los dispositivos permanecen visibles aunque alguno deje de responder. El color de cada elemento indica su estado:

- Verde: dispositivo o enlace disponible.
- Rojo: dispositivo, adyacencia OSPF o sesión MPLS LDP caída.
- Amarillo: todavía no existen datos suficientes para determinar el estado.

Los dispositivos utilizados por la página se muestran en la Tabla **??**.

4.6.4. Métricas utilizadas

La página consulta tres tipos principales de información.

Disponibilidad de dispositivos

La disponibilidad de los dispositivos se obtiene de las gráficas generadas por el colector de ping de Netdata. Se utiliza el porcentaje de pérdida de paquetes para clasificar cada equipo como disponible o no disponible.

Los identificadores siguen la siguiente estructura:

```
ping_dispositivos_huawei.host_packet_loss_AR651-A
ping_dispositivos_huawei.host_packet_loss_S6730H-C
ping_dispositivos_huawei.host_packet_loss_S6730H-D
ping_dispositivos_huawei.host_packet_loss_AR651-B
```

Si la pérdida de paquetes alcanza el 100 %, el dispositivo se muestra en color rojo, pero no se elimina del diagrama.

Estado de OSPF y MPLS LDP

Los estados permanentes de los enlaces se obtienen de las métricas creadas previamente mediante el colector `charts.d`.

Para OSPF se utilizan:

```
huawei_routing.ospf_a_c
huawei_routing.ospf_c_d
huawei_routing.ospf_d_b
```

Para MPLS LDP se utilizan:

```
huawei_routing.ldp_a_c
huawei_routing.ldp_c_d
huawei_routing.ldp_d_b
```

Cada gráfica contiene la dimensión:

```
available
```

La interpretación es la siguiente:

- `available = 1`: ambos extremos reportan el estado correcto.
- `available = 0`: la fila SNMP desapareció, el estado cambió o alguno de los extremos dejó de responder.

Para OSPF, el estado correcto corresponde a `Full`. Para MPLS LDP, corresponde a `Operational`.

Tráfico de los enlaces

El tráfico se obtiene de las gráficas SNMP de cada interfaz. Cada gráfica contiene las dimensiones `in` y `out`.

Por ejemplo, para el enlace entre AR651-A y S6730H-C se consultan:

```
snmp_AR651-A.snmp_device_prof_ifTraffic_GigabitEthernet0/0/1

snmp_S6730H-C.snmp_device_prof_ifTraffic_GigabitEthernet0/0/1
```

Las demás interfaces utilizan el mismo contexto:

```
snmp.device_prof_ifTraffic
```

Al colocar el cursor sobre un enlace, la página muestra:

- Estado de la adyacencia OSPF.
- Estado de la sesión MPLS LDP.
- Nombre de la interfaz en ambos extremos.
- Tráfico entrante en cada extremo.
- Tráfico saliente en cada extremo.
- Valores expresados en Mbit/s.

4.6.5. Funcionamiento del servidor Python

El archivo `server.py` utiliza únicamente módulos incluidos en Python, por lo que no fue necesario instalar bibliotecas externas.

El servidor escucha conexiones en todas las interfaces de la Raspberry Pi, utilizando el puerto 8080:

```
server = ThreadingHTTPServer(("0.0.0.0", 8080), Handler)
```

Las solicitudes cuyo destino comienza con `/netdata/` son reenviadas hacia la API local de Netdata:

```
NETDATA = "http://127.0.0.1:19999"
target = NETDATA + self.path[len("/netdata"):]
```

Este mecanismo funciona como un proxy local y evita problemas de seguridad del navegador relacionados con solicitudes entre diferentes orígenes.

4.6.6. Actualización automática de la página

El archivo `index.html` contiene código JavaScript que consulta periódicamente el servidor Python. La actualización se ejecuta cada 15 segundos:

```
refresh();
setInterval(refresh, 15000);
```

En cada actualización se realizan las siguientes operaciones:

1. Se consulta el porcentaje de pérdida de paquetes de cada dispositivo.
2. Se consulta el estado OSPF de los tres enlaces.
3. Se consulta el estado MPLS LDP de los tres enlaces.
4. Se consulta el tráfico entrante y saliente de las interfaces.
5. Se actualizan los colores y textos de la topología.
6. Se actualiza el contenido mostrado al colocar el cursor sobre un enlace.

4.6.7. Instalación de la aplicación

El archivo comprimido se copió hacia la Raspberry Pi mediante SCP:

```
scp netdata-topology.tar.gz \
telecom@10.166.114.156:/tmp/
```

Posteriormente, se extrajo el contenido dentro del directorio `/opt`:

```
sudo tar -xzf \
/tmp/netdata-topology.tar.gz \
-C /opt
```

El resultado fue el directorio:

```
/opt/netdata-topology
```

Se asignó el propietario `root` y el grupo `netdata`:

```
sudo chown -R root:netdata \
/opt/netdata-topology
```

También se establecieron permisos restrictivos para el directorio y sus archivos:

```
sudo chmod 750 \
/opt/netdata-topology

sudo chmod 640 \
/opt/netdata-topology/index.html

sudo chmod 750 \
/opt/netdata-topology/server.py
```

4.6.8. Creación del servicio de systemd

Para iniciar automáticamente la página junto con el sistema operativo se instaló el archivo de servicio:

```
sudo cp \
/opt/netdata-topology/netdata-topology.service \
/etc/systemd/system/
```

El servicio contiene la siguiente configuración principal:

```
[Unit]
Description=Topologia personalizada basada en Netdata
After=network-online.target netdata.service
Wants=network-online.target

[Service]
Type=simple
User=netdata
Group=netdata
WorkingDirectory=/opt/netdata-topology
ExecStart=/usr/bin/python3 /opt/netdata-topology/server.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Se recargó la configuración de systemd:

```
sudo systemctl daemon-reload
```

Finalmente, se habilitó e inició el servicio:

```
sudo systemctl enable --now netdata-topology
```

El estado del servicio se verificó con:

```
sudo systemctl status \
netdata-topology \
--no-pager
```

También se verificó localmente la respuesta del servidor:

```
curl -I http://127.0.0.1:8080
```

4.6.9. Acceso a la topología

La topología se encuentra disponible desde un navegador web mediante la dirección IP de la Raspberry Pi y el puerto 8080:

```
http://10.166.114.156:8080
```

La página se actualiza automáticamente, por lo que no es necesario recargar manualmente el navegador después de apagar o restaurar un dispositivo o enlace.

4.6.10. Consideraciones de seguridad

El servidor personalizado no incorpora autenticación propia. Por esta razón, el puerto 8080 debe estar disponible únicamente desde la red administrativa y no debe publicarse directamente hacia Internet.

4.7. Habilitación del monitoreo de OSPF y MPLS mediante SNMP

Para supervisar el estado de los protocolos OSPF y MPLS desde Netdata, fue necesario comprobar que los dispositivos Huawei expusieran esta información mediante SNMP.

Aunque OSPF y MPLS se encontraban funcionando correctamente, esto no garantiza que sus estados internos puedan consultarse automáticamente mediante SNMP. En los equipos Huawei, la información del proceso OSPF debe vincularse explícitamente con la MIB estándar de OSPF. [20]

4.7.1. Vinculación del proceso OSPF con SNMP

Huawei permite ejecutar varios procesos OSPF simultáneamente. Por esta razón, el dispositivo necesita conocer cuál de estos procesos debe exponer mediante la MIB estándar `OSPF-MIB`. [21]

Primero se identificó el proceso OSPF utilizado en cada dispositivo:

```
display ospf brief
```

También se utilizó el siguiente comando para comprobar la configuración:

```
display current-configuration | include ospf
```

En la topología implementada se utilizó el proceso OSPF número 1. Por lo tanto, en cada router y switch se ejecutó:

```
system-view
ospf mib-binding 1
return
save
```

El comando `ospf mib-binding 1` vincula el proceso OSPF número 1 con la MIB pública de OSPF. Después de realizar esta vinculación, el dispositivo puede responder a las consultas SNMP relacionadas con las interfaces, áreas y vecinos OSPF.

El comando debe ejecutarse desde la vista global del equipo:

```
[AR651-A] ospf mib-binding 1
```

Si se utiliza un proceso OSPF diferente, el número 1 debe sustituirse por el identificador correspondiente:

```
ospf mib-binding <process-id>
```

Finalmente, se guardó la configuración para evitar que se perdiera después de reiniciar los dispositivos:

```
save
```

4.7.2. Importancia de la vinculación OSPF

Antes de realizar la vinculación, la consulta de la tabla de vecinos OSPF no devolvía información, aunque las adyacencias estaban establecidas y el protocolo funcionaba correctamente.

Esto se debe a que el funcionamiento del protocolo de enrutamiento y su exposición mediante SNMP son procesos diferentes. El comando `ospf mib-binding` no habilita OSPF ni modifica las adyacencias existentes; únicamente permite que un sistema de gestión consulte su estado mediante la MIB estándar.

Después de realizar la vinculación, se pudo consultar el objeto `ospfNbrState`, cuyo OID es:

```
1.3.6.1.2.1.14.10.1.6
```

La consulta utilizada para los routers y switches con autenticación SHA-256 y cifrado AES (Recordar que estos comandos son desde el servidor de Netdata, en este caso, Raspberry pi) fue:

```
snmpwalk -v3 \
-l authPriv \
-u netdata-user \
-a SHA-256 \
-A 'NetAuth2026' \
-x AES \
-X 'NetPriv2026' \
1.1.1.1 \
1.3.6.1.2.1.14.10.1.6
```

4.7.3. Estados de los vecinos OSPF

El objeto `ospfNbrState` representa el estado actual de cada vecino OSPF. Los valores posibles se presentan en la Tabla 4.2.

Cuadro 4.2: Estados de una adyacencia OSPF

Valor	Estado	Descripción
1	Down	No se han recibido paquetes Hello del vecino durante el intervalo correspondiente.
2	Attempt	El dispositivo intenta establecer comunicación con un vecino configurado manualmente. Este estado se utiliza principalmente en redes NBMA.
3	Init	Se han recibido paquetes Hello del vecino, pero el router local todavía no aparece identificado dentro de dichos paquetes.
4	Two-Way	Existe comunicación bidireccional entre los routers. Ambos dispositivos se reconocen mediante los paquetes Hello.
5	Exchange Start	Los vecinos negocian cuál dispositivo actuará como maestro durante el intercambio de información de la base de datos.
6	Exchange	Los routers intercambian paquetes de descripción de base de datos o DBD.
7	Loading	Los dispositivos solicitan y reciben información faltante de la base de datos de estado de enlace.
8	Full	La adyacencia está completamente establecida y las bases de datos de estado de enlace se encuentran sincronizadas.

El estado esperado durante la operación normal es:

```
INTEGER: 8
```

Por lo tanto, el valor 8 indica que el vecino se encuentra en estado *Full*. Cualquier estado inferior representa que la adyacencia todavía está en proceso de formación o que existe algún problema de comunicación.

4.7.4. Consulta del estado MPLS LDP

Para verificar el estado de MPLS se consultó la tabla de sesiones del protocolo LDP (*Label Distribution Protocol*).

El objeto utilizado fue `mplsLdpSessionState`, cuyo OID es:

```
1.3.6.1.2.1.10.166.4.1.3.3.1.2
```

En los routers y switches se utilizó la siguiente consulta:

```
snmpwalk -v3 \
-l authPriv \
-u netdata-user \
-a SHA-256 \
-A 'NetAuth2026' \
-x AES \
-X 'NetPriv2026' \
1.1.1.1 \
1.3.6.1.2.1.10.166.4.1.3.3.1.2
```

4.7.5. Estados de las sesiones MPLS LDP

Los posibles estados de una sesión LDP se presentan en la Tabla 4.3.

Cuadro 4.3: Estados de una sesión MPLS LDP

Valor	Estado	Descripción
1	Nonexistent	La sesión LDP no existe o fue eliminada.
2	Initialized	La sesión fue inicializada y comenzó el proceso de establecimiento.
3	Open Received	Se recibió un mensaje de inicialización LDP desde el dispositivo vecino.
4	Open Sent	Se envió un mensaje de inicialización hacia el vecino y se espera la continuación de la negociación.
5	Operational	La sesión está completamente establecida y puede intercambiar etiquetas MPLS.

Durante las pruebas se obtuvo el siguiente resultado:

```
INTEGER: 5
```

El valor 5 indica que la sesión LDP está en estado *Operational*. En este estado, los dispositivos pueden intercambiar etiquetas y construir los caminos conmutados por etiquetas o LSP (*Label Switched Paths*).

4.7.6. Interpretación conjunta de los estados

La supervisión conjunta de las interfaces, las adyacencias OSPF y las sesiones MPLS LDP permite identificar con mayor precisión el origen de una falla.

Cuadro 4.4: Interpretación conjunta de los estados de la red

Interfaz	OSPF	LDP	Interpretación
Up	Full	Operational	El enlace físico, el protocolo OSPF y la sesión MPLS funcionan correctamente.
Up	Distinto de Full	Operational o inactivo	La interfaz funciona, pero existe una falla o proceso de convergencia en OSPF.
Up	Full	Distinto de Operational	La conectividad IP y OSPF funcionan, pero existe un problema específico en MPLS LDP.
Down	Down	Nonexistent	La interfaz o el enlace físico se encuentra caído, provocando también la pérdida de OSPF y MPLS.

Esta separación permite que el sistema de monitoreo no se limite a indicar que un dispositivo está disponible, sino que identifique si la falla corresponde al enlace físico, al protocolo de enrutamiento OSPF o al intercambio de etiquetas MPLS LDP.

Para comprobar los índices exactos que capturas los dispositivos se usa:

```
for equipo in \
  "AR651-A 1.1.1.1 AES" \
  "S6730H-C 2.2.2.2 AES" \
  "S6730H-D 3.3.3.3 AES" \
```

```
  "AR651-B 4.4.4.4 AES"
do
  set -- $equipo
  nombre=$1
  ip=$2
  cifrado=$3

  echo "========== $nombre - OSPF =========="

  snmpwalk -On -v3 \
    -l authPriv \
    -u netdata-user \
    -a SHA-256 \
    -A 'NetAuth2026' \
    -x "$cifrado" \
    -X 'NetPriv2026' \
    "$ip" \
    1.3.6.1.2.1.14.10.1.6

  echo "========== $nombre - MPLS LDP =========="

  snmpwalk -On -v3 \
    -l authPriv \
    -u netdata-user \
    -a SHA-256 \
    -A 'NetAuth2026' \
    -x "$cifrado" \
    -X 'NetPriv2026' \
    "$ip" \
    1.3.6.1.2.1.10.166.4.1.3.3.1.2
done
```

```
========== AR651-A - OSPF ==========
.1.3.6.1.2.1.14.10.1.6.10.0.12.2.0 = INTEGER: 8
========== AR651-A - MPLS LDP ==========
.1.3.6.1.2.1.10.166.4.1.3.3.1.2.1.1.1.1.0.0.4.2.2.2.2.0.0 = INTEGER: 5
========== S6730H-C - OSPF ==========
.1.3.6.1.2.1.14.10.1.6.10.0.12.1.0 = INTEGER: 8
.1.3.6.1.2.1.14.10.1.6.10.0.23.2.0 = INTEGER: 8
========== S6730H-C - MPLS LDP ==========
.1.3.6.1.2.1.10.166.4.1.3.3.1.2.2.2.2.2.0.0.6.1.1.1.1.0.0 = INTEGER: 5
.1.3.6.1.2.1.10.166.4.1.3.3.1.2.2.2.2.2.0.0.7.3.3.3.3.0.0 = INTEGER: 5
========== S6730H-D - OSPF ==========
.1.3.6.1.2.1.14.10.1.6.10.0.23.1.0 = INTEGER: 8
.1.3.6.1.2.1.14.10.1.6.10.0.34.2.0 = INTEGER: 8
========== S6730H-D - MPLS LDP ==========
.1.3.6.1.2.1.10.166.4.1.3.3.1.2.3.3.3.3.0.0.6.2.2.2.2.0.0 = INTEGER: 5
.1.3.6.1.2.1.10.166.4.1.3.3.1.2.3.3.3.3.0.0.7.4.4.4.4.0.0 = INTEGER: 5
========== AR651-B - OSPF ==========
.1.3.6.1.2.1.14.10.1.6.10.0.34.1.0 = INTEGER: 8
========== AR651-B - MPLS LDP ==========
.1.3.6.1.2.1.10.166.4.1.3.3.1.2.4.4.4.4.0.0.4.3.3.3.3.0.0 = INTEGER: 5
```

Figura 4.13: Resultado de datos exactos que muestran los equipos

4.7.7. Crear el perfil de enrutamiento en Netdata

- Crear el perfil de enrutamiento

```
sudo nano /etc/netdata/go.d/snmp.profiles/huawei-routing-status.yaml
```

- Pegar lo siguiente:

```
selector:
  - sysobjectid:
      include:
        - "1.3.6.1.4.1.2011.*"

metrics:
  # ============================================================
  # Estado de vecinos OSPF
  # Estado correcto: full = 8
  # ============================================================
  - MIB: OSPF-MIB
    table:
      OID: 1.3.6.1.2.1.14.10
      name: ospfNbrTable

    symbols:
      - OID: 1.3.6.1.2.1.14.10.1.6
        name: ospfNbrState
        metric_type: gauge

        chart_meta:
          description: Estado de la adyacencia OSPF
          family: "Routing/OSPF/Neighbors"
          unit: "state"

        mapping:
          items:
            1: down
            2: attempt
            3: init
            4: twoWay
            5: exchangeStart
            6: exchange
            7: loading
            8: full

    metric_tags:
      - tag: neighbor
        symbol:
          name: ospfNbrIpAddr
          format: ip_address
        index_transform:
          - start: 0
            end: 3

  # ============================================================
  # Estado de sesiones MPLS LDP
  # Estado correcto: operational = 5
  # ============================================================
  - MIB: MPLS-LDP-STD-MIB
    table:
      OID: 1.3.6.1.2.1.10.166.4.1.3.3
      name: mplsLdpSessionTable

    symbols:
      - OID: 1.3.6.1.2.1.10.166.4.1.3.3.1.2
```

```
        name: mplsLdpSessionState
        metric_type: gauge

        chart_meta:
          description: Estado de la sesi\'on MPLS LDP
          family: "Routing/MPLS/LDP Sessions"
          unit: "state"

        mapping:
          items:
            1: nonexistent
            2: initialized
            3: openrec
            4: opensent
            5: operational

    metric_tags:
      - tag: peer
        symbol:
          name: mplsLdpPeerAddress
          format: ip_address
        index_transform:
          - start: 7
            drop_right: 2
```

Netdata exige que las métricas provenientes de tablas tengan al menos una etiqueta que identifique cada fila. Aquí las etiquetas neighbor y peer se extraen del índice del OID, en lugar de consultar otra columna SNMP. El mapping convierte los números en dimensiones legibles, como full, down y operational. [22]

- Se establecen permisos:

```
sudo chown root:netdata \
  /etc/netdata/go.d/snmp.profiles/huawei-routing-status.yaml

sudo chmod 640 \
  /etc/netdata/go.d/snmp.profiles/huawei-routing-status.yaml

sudo chmod 750 \
  /etc/netdata/go.d/snmp.profiles
```

- Luego de esto, se debe de reiniciar Netdata:

```
sudo systemctl restart netdata
```

- Antes de crear las alertas, necesitamos confirmar los nombres exactos de las dimensiones generadas (full, down, operational, etc.). Ejecutar:

```
python3 - <<'PY'
import json
import urllib.request
```

```
nodes = ["AR651-A", "S6730H-C", "S6730H-D", "AR651-B"]

for node in nodes:
    url = f"http://127.0.0.1:19999/host/{node}/api/v1/charts"

    with urllib.request.urlopen(url) as response:
        charts = json.load(response).get("charts", {})

    print(f"\n========== {node} ==========")

    for chart in charts.values():
        chart_id = chart.get("id", "")

        # Solo las graficas limpias creadas por el perfil nuevo.
        if (
            "ospfNbrState" in chart_id
            or "mplsLdpSessionState" in chart_id
        ):
            dimensions = list(chart.get("dimensions", {}).keys())

            print("ID:", chart_id)
            print("CONTEXT:", chart.get("context"))
            print("DIMENSIONS:", dimensions)
            print("LABELS:", chart.get("chart_labels", {}))
            print()
PY
```

```
========== AR651-A ==========
ID: snmp_AR651-A.snmp_device_prof_ospfNbrState_10_0_12_2_10_0_12_2
CONTEXT: snmp.device_prof_ospfNbrState
DIMENSIONS: ['init', 'twoWay', 'exchangeStart', 'exchange', 'loading', 'full', 'down', 'attempt']
LABELS: {'neighbor_id': '10.0.12.2', 'neighbor_ip': '10.0.12.2', 'model': 'AR2220', 'address': '1.1.1.1', 'vendor': 'Huawei', '_collect_module': 'snmp', '_col
lect_job': 'AR651-A', 'sysName': 'AR651-A', 'device_type': 'Router', '_collect_plugin': 'go.d'}

ID: snmp_AR651-A.snmp_device_prof_ospfNbrState_10_0_12_2
CONTEXT: snmp.device_prof_ospfNbrState
DIMENSIONS: ['full', 'down', 'attempt', 'init', 'twoWay', 'exchangeStart', 'exchange', 'loading']
LABELS: {'neighbor': '10.0.12.2', 'model': 'AR2220', 'address': '1.1.1.1', 'vendor': 'Huawei', '_collect_module': 'snmp', '_collect_job': 'AR651-A', 'sysName'
: 'AR651-A', 'device_type': 'Router', '_collect_plugin': 'go.d'}

ID: snmp_AR651-A.snmp_device_prof_mplsLdpSessionState_2_2_2_2
CONTEXT: snmp.device_prof_mplsLdpSessionState
DIMENSIONS: ['nonexistent', 'initialized', 'openrec', 'opensent', 'operational']
LABELS: {'peer': '2.2.2.2', 'model': 'AR2220', 'address': '1.1.1.1', 'vendor': 'Huawei', '_collect_module': 'snmp', '_collect_job': 'AR651-A', 'sysName': 'AR6
51-A', 'device_type': 'Router', '_collect_plugin': 'go.d'}
```

Figura 4.14: Ejemplo de salida con Router A

- Crear las alertas.

```
sudo nano /etc/netdata/health.d/huawei_routing.conf
```

Pegar el contenido:

```
# ============================================================
# Estado permanente de enlaces OSPF esperados
#
# Contexto generado por:
#   huawei_routing.chart.sh
#
# Valor:
```

```
#   1 = ambos extremos se encuentran en Full
#   0 = estado incorrecto, fila ausente o equipo inaccesible
# ============================================================

template: huawei_ospf_expected_link_down
      on: huawei.routing.ospf_link
   class: Availability
    type: Network
component: OSPF
    calc: $available
   units: state
   every: 30s
    crit: $this < 1
   delay: up 30s down 30s
 summary: Enlace OSPF ${label:link} caido
    info: La adyacencia OSPF ${label:link} dejo de estar en estado
    Full o alguno de sus extremos no responde por SNMP.
      to: sysadmin


# ============================================================
# Estado permanente de sesiones MPLS LDP esperadas
#
# Valor:
#   1 = ambos extremos se encuentran en Operational
#   0 = estado incorrecto, fila ausente o equipo inaccesible
# ============================================================

template: huawei_mpls_ldp_expected_link_down
      on: huawei.routing.ldp_link
   class: Availability
    type: Network
component: MPLS
    calc: $available
   units: state
   every: 30s
    crit: $this < 1
   delay: up 30s down 30s
 summary: Sesion MPLS LDP ${label:link} caida
    info: La sesion MPLS LDP ${label:link} dejo de estar en estado
    Operational o alguno de sus extremos no responde por SNMP.
      to: sysadmin
```

La ventana de un minuto tolera una muestra aislada, mientras que el retraso de 30 segundos evita alertas por una transición demasiado breve. Las variables label:neighbor, label:peer y label:sysName se sustituyen por las etiquetas de cada gráfica.

- Aplicar permisos:

```
 sudo chown root:netdata \
  /etc/netdata/health.d/huawei_routing.conf

sudo chmod 640 \
  /etc/netdata/health.d/huawei_routing.conf
```

- Recargar las alertas:

```
sudo netdatacli reload-health
```

- Comprobar:

```
python3 - <<'PY'
import json
import urllib.request

nodes = ["AR651-A", "S6730H-C", "S6730H-D", "AR651-B"]

for node in nodes:
    url = f"http://127.0.0.1:19999/host/{node}/api/v1/alarms?all"

    print(f"\n========== {node} ==========")

    with urllib.request.urlopen(url) as response:
        alarms = json.load(response).get("alarms", {})

    found = False

    for alarm in alarms.values():
        name = alarm.get("name", "")

        if name in [
            "huawei_ospf_neighbor_not_full",
            "huawei_mpls_ldp_session_not_operational"
        ]:
            found = True

            print(
                "ALERTA:", name,
                "\nCHART:", alarm.get("chart"),
                "\nESTADO:", alarm.get("status"),
                "\nVALOR:", alarm.get("value"),
                "\n"
            )

    if not found:
        print("No hay instancias activas.")
PY
```

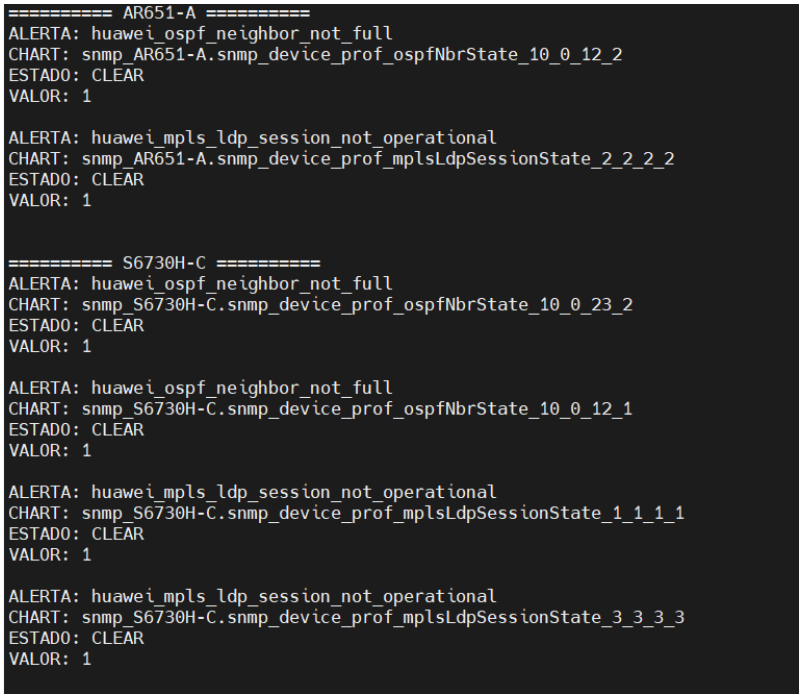


Figura 4.15: Ejemplo de salida con Router A y switch C

- Ahora se debe de crear comprobaciones permanentes, esto porque a la hora de que se caiga un enlace OSPF, los dispositivos simplemente los borran de su tabla de enrutamiento, entonces netdata, nunca recibe un error, para esto se realiza un nuevo colector de netdata, asi como las alertas:

```
sudo mkdir -p /etc/netdata/charts.d
sudo nano /etc/netdata/charts.d/huawei_routing.conf
```

En este archivo pegar:

```
# Frecuencia de comprobacion
huawei_routing_update_every=30

# Credenciales SNMPv3
HUAWEI_SNMP_USER="netdata-user"
HUAWEI_AUTH_PROTOCOL="SHA"
HUAWEI_AUTH_KEY="NetAuth2026"
HUAWEI_PRIVACY_KEY="NetPriv2026"

# Direcciones de administracion
AR651_A_HOST="1.1.1.1"
S6730H_C_HOST="2.2.2.2"
S6730H_D_HOST="3.3.3.3"
AR651_B_HOST="4.4.4.4"

# Cifrado empleado en eNSP
AR651_A_PRIVACY="AES"
S6730H_C_PRIVACY="DES"
S6730H_D_PRIVACY="DES"
AR651_B_PRIVACY="AES"
```

- Configurar los permisos:

```
sudo chown root:netdata \
  /etc/netdata/charts.d/huawei_routing.conf

sudo chmod 640 \
  /etc/netdata/charts.d/huawei_routing.conf
```

- Ahora se debe de crear el nuevo colector:

```
sudo nano \
  /usr/libexec/netdata/charts.d/huawei_routing.chart.sh
```

En este archivo pegar:

```
# shellcheck shell=bash

# ============================================================
# Comprobacion permanente de enlaces OSPF y MPLS LDP Huawei
#
# Valores publicados:
#   1 = ambos extremos se encuentran en el estado esperado
#   0 = estado incorrecto, fila ausente o equipo inaccesible
# ============================================================

huawei_routing_update_every="${huawei_routing_update_every:-30}"
huawei_routing_priority=90000


# ------------------------------------------------------------
# Consulta un OID y comprueba su valor.
#
# Parametros:
#   $1 = dispositivo
#   $2 = protocolo de privacidad: AES o DES
#   $3 = OID completo
#   $4 = valor esperado
# ------------------------------------------------------------

huawei_routing_oid_is()
{
    local host="$1"
    local privacy="$2"
    local oid="$3"
    local expected="$4"
    local value

    value="$(
        snmpget \
          -v3 \
          -l authPriv \
          -u "${HUAWEI_SNMP_USER}" \
          -a "${HUAWEI_AUTH_PROTOCOL}" \
          -A "${HUAWEI_AUTH_KEY}" \
          -x "${privacy}" \
```

```
            -X "${HUAWEI_PRIVACY_KEY}" \
            -t 2 \
            -r 0 \
            -Oqv \
            "${host}" \
            "${oid}" \
            2>/dev/null
    )"

    value="${value//[[:space:]]/}"

    [[ "${value}" == "${expected}" ]]
}


# ----------------------------------------------------------------
# Devuelve 1 si los dos extremos estan correctos.
# De lo contrario, devuelve 0.
# ----------------------------------------------------------------

huawei_routing_pair_state()
{
    if huawei_routing_oid_is "$1" "$2" "$3" "$4" &&
       huawei_routing_oid_is "$5" "$6" "$7" "$8"
    then
        echo 1
    else
        echo 0
    fi
}


huawei_routing_check()
{
    require_cmd snmpget || return 1

    [[ -n "${HUAWEI_SNMP_USER}" ]] || return 1
    [[ -n "${HUAWEI_AUTH_KEY}" ]] || return 1
    [[ -n "${HUAWEI_PRIVACY_KEY}" ]] || return 1

    return 0
}


huawei_routing_create()
{
    cat <<EOF
CHART huawei_routing.ospf_a_c '' 'OSPF: AR651-A - S6730H-C' 'state' '
    Routing/OSPF' 'huawei.routing.ospf_link' line 90000 ${
    huawei_routing_update_every}
CLABEL protocol OSPF 1
CLABEL link AR651-A--S6730H-C
CLABEL_COMMIT
DIMENSION available 'available' absolute 1 1

CHART huawei_routing.ospf_c_d '' 'OSPF: S6730H-C - S6730H-D' 'state' '
    Routing/OSPF' 'huawei.routing.ospf_link' line 90001 ${
    huawei_routing_update_every}
```

```
CLABEL protocol OSPF 1
CLABEL link S6730H-C--S6730H-D
CLABEL_COMMIT
DIMENSION available 'available' absolute 1 1

CHART huawei_routing.ospf_d_b '' 'OSPF: S6730H-D - AR651-B' 'state' '
    Routing/OSPF' 'huawei.routing.ospf_link' line 90002 ${
    huawei_routing_update_every}
CLABEL protocol OSPF 1
CLABEL link S6730H-D--AR651-B
CLABEL_COMMIT
DIMENSION available 'available' absolute 1 1

CHART huawei_routing.ldp_a_c '' 'MPLS LDP: AR651-A - S6730H-C' 'state'
     'Routing/MPLS LDP' 'huawei.routing.ldp_link' line 90003 ${
    huawei_routing_update_every}
CLABEL protocol MPLS_LDP 1
CLABEL link AR651-A--S6730H-C
CLABEL_COMMIT
DIMENSION available 'available' absolute 1 1

CHART huawei_routing.ldp_c_d '' 'MPLS LDP: S6730H-C - S6730H-D' 'state
    ' 'Routing/MPLS LDP' 'huawei.routing.ldp_link' line 90004 ${
    huawei_routing_update_every}
CLABEL protocol MPLS_LDP 1
CLABEL link S6730H-C--S6730H-D
CLABEL_COMMIT
DIMENSION available 'available' absolute 1 1

CHART huawei_routing.ldp_d_b '' 'MPLS LDP: S6730H-D - AR651-B' 'state'
     'Routing/MPLS LDP' 'huawei.routing.ldp_link' line 90005 ${
    huawei_routing_update_every}
CLABEL protocol MPLS_LDP 1
CLABEL link S6730H-D--AR651-B
CLABEL_COMMIT
DIMENSION available 'available' absolute 1 1
EOF

    return 0
}


huawei_routing_update()
{
    local microseconds="$1"

    local ospf_a_c
    local ospf_c_d
    local ospf_d_b
    local ldp_a_c
    local ldp_c_d
    local ldp_d_b

    # ========================================================
    # OSPF: valor esperado 8 = Full
    # ========================================================

    ospf_a_c="$(
```

```
    huawei_routing_pair_state \
      "${AR651_A_HOST}" "${AR651_A_PRIVACY}" \
      "1.3.6.1.2.1.14.10.1.6.10.0.12.2.0" "8" \
      "${S6730H_C_HOST}" "${S6730H_C_PRIVACY}" \
      "1.3.6.1.2.1.14.10.1.6.10.0.12.1.0" "8"
)"

ospf_c_d="$(
    huawei_routing_pair_state \
      "${S6730H_C_HOST}" "${S6730H_C_PRIVACY}" \
      "1.3.6.1.2.1.14.10.1.6.10.0.23.2.0" "8" \
      "${S6730H_D_HOST}" "${S6730H_D_PRIVACY}" \
      "1.3.6.1.2.1.14.10.1.6.10.0.23.1.0" "8"
)"

ospf_d_b="$(
    huawei_routing_pair_state \
      "${S6730H_D_HOST}" "${S6730H_D_PRIVACY}" \
      "1.3.6.1.2.1.14.10.1.6.10.0.34.2.0" "8" \
      "${AR651_B_HOST}" "${AR651_B_PRIVACY}" \
      "1.3.6.1.2.1.14.10.1.6.10.0.34.1.0" "8"
)"

# ============================================================
# MPLS LDP: valor esperado 5 = Operational
# ============================================================

ldp_a_c="$(
    huawei_routing_pair_state \
      "${AR651_A_HOST}" "${AR651_A_PRIVACY}" \
      "1.3.6.1.2.1.10.166.4.1.3.3.1.2.1.1.1.1.0.0.4.2.2.2.2.0.0" "
5" \
      "${S6730H_C_HOST}" "${S6730H_C_PRIVACY}" \
      "1.3.6.1.2.1.10.166.4.1.3.3.1.2.2.2.2.2.0.0.6.1.1.1.1.0.0" "
5"
)"

ldp_c_d="$(
    huawei_routing_pair_state \
      "${S6730H_C_HOST}" "${S6730H_C_PRIVACY}" \
      "1.3.6.1.2.1.10.166.4.1.3.3.1.2.2.2.2.2.0.0.7.3.3.3.3.0.0" "
5" \
      "${S6730H_D_HOST}" "${S6730H_D_PRIVACY}" \
      "1.3.6.1.2.1.10.166.4.1.3.3.1.2.3.3.3.3.0.0.6.2.2.2.2.0.0" "
5"
)"

ldp_d_b="$(
    huawei_routing_pair_state \
      "${S6730H_D_HOST}" "${S6730H_D_PRIVACY}" \
      "1.3.6.1.2.1.10.166.4.1.3.3.1.2.3.3.3.3.0.0.7.4.4.4.4.0.0" "
5" \
      "${AR651_B_HOST}" "${AR651_B_PRIVACY}" \
      "1.3.6.1.2.1.10.166.4.1.3.3.1.2.4.4.4.4.0.0.4.3.3.3.3.0.0" "
5"
)"

cat <<EOF
```

```
BEGIN huawei_routing.ospf_a_c ${microseconds}
SET available = ${ospf_a_c}
END

BEGIN huawei_routing.ospf_c_d ${microseconds}
SET available = ${ospf_c_d}
END

BEGIN huawei_routing.ospf_d_b ${microseconds}
SET available = ${ospf_d_b}
END

BEGIN huawei_routing.ldp_a_c ${microseconds}
SET available = ${ldp_a_c}
END

BEGIN huawei_routing.ldp_c_d ${microseconds}
SET available = ${ldp_c_d}
END

BEGIN huawei_routing.ldp_d_b ${microseconds}
SET available = ${ldp_d_b}
END
EOF

    return 0
}
```

- Aplicar permisos al colector:

```
sudo chown root:netdata \
  /usr/libexec/netdata/charts.d/huawei_routing.chart.sh

sudo chmod 755 \
  /usr/libexec/netdata/charts.d/huawei_routing.chart.sh
```

- Para que netdata lo integre, se activa:

```
sudo nano /etc/netdata/charts.d.conf
```

  En este archivo, se debe de colocar yes

```
 huawei_routing="yes"
```

- Se debe de habilitar el proceso charts.d.plugin globalmente, para descargar los archivos complementarios necesarios, el mismo netdata en el archivo:

```
sudo nano /etc/netdata/netdata.conf
```

- Recomienda actualizarlo a la última versión:

```
wget -O /etc/netdata/netdata.conf http://localhost:19999/netdata.
    conf
```

- Luego de actulizarlo, se debe de buscar la lista de [plugins], y debe de quedar descomentado el servicio:

```
[plugins]
        # idlejitter = yes
        # netdata pulse = yes
        # profile = no
        # tc = yes
        # diskspace = yes
        # proc = yes
        # cgroups = yes
        # timex = yes
        # enable running new plugins = yes
        # check for new plugins every = 1m
        # slabinfo = no
        # go.d = yes
        # systemd-units = yes
        # systemd-journal = yes
        # statsd = yes
        # perf = yes
        # apps = yes
        # network-viewer = yes
        # nfacct = yes
        # scripts.d = yes
        # debugfs = yes
        charts.d = yes
        # python.d = yes
        # ioping = yes
        # otel = yes
```

Figura 4.16: Servicio de charts.d habilitado

- Reiniciar el agente de netdata para aplicar los cambios:

```
sudo systemctl restart netdata
```

- Se comprobó su funcionamiento apagando la interfaz que conecta switch D con router B, y en netdata aparece:

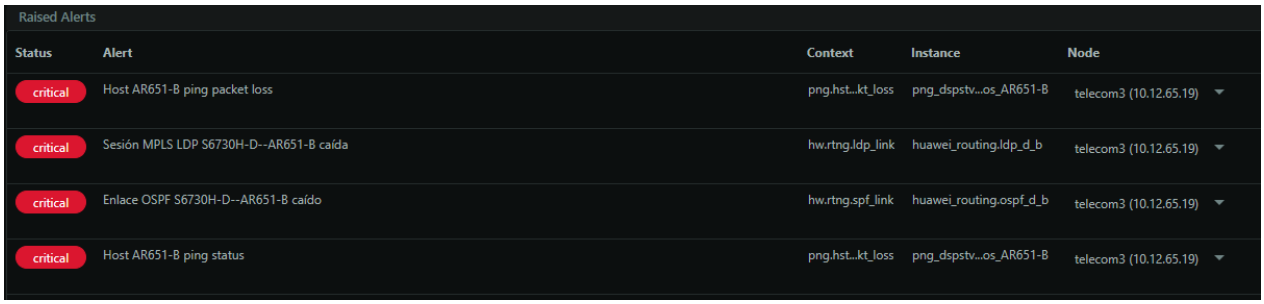


Figura 4.17: Funcionamiento de alerta de caída de OSPF y MPLS

Capítulo 5

Anexo

5.1. Anexo

```
PS C:\WINDOWS\system32> iperf3 -c 20.20.20.1 -u -b 500M -t 3600 -i 1
Connecting to host 20.20.20.1, port 5201
[  5] local 10.10.10.1 port 57084 connected to 20.20.20.1 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  59.7 MBytes   499 Mbits/sec  42847
[  5]   1.00-2.01   sec  59.5 MBytes   496 Mbits/sec  42758
[  5]   2.01-3.01   sec  60.0 MBytes   500 Mbits/sec  43057
[  5]   3.01-4.01   sec  59.4 MBytes   502 Mbits/sec  42654
[  5]   4.01-5.01   sec  58.9 MBytes   495 Mbits/sec  42331
[  5]   5.01-6.01   sec  60.8 MBytes   505 Mbits/sec  43640
[  5]   6.01-7.01   sec  59.4 MBytes   501 Mbits/sec  42656
[  5]   7.01-8.00   sec  59.0 MBytes   497 Mbits/sec  42365
[  5]   8.00-9.01   sec  59.8 MBytes   500 Mbits/sec  42929
[  5]   9.01-10.00  sec  59.5 MBytes   502 Mbits/sec  42711
[  5]  10.00-11.01  sec  60.4 MBytes   502 Mbits/sec  43373
[  5]  11.01-12.01  sec  59.1 MBytes   498 Mbits/sec  42445
[  5]  12.01-13.00  sec  46.0 MBytes   387 Mbits/sec  33018
[  5]  13.00-14.01  sec  53.7 MBytes   446 Mbits/sec  38576
[  5]  14.01-15.00  sec  68.7 MBytes   581 Mbits/sec  49325
[  5]  15.00-16.01  sec  68.7 MBytes   572 Mbits/sec  49331
[  5]  16.01-17.01  sec  60.7 MBytes   512 Mbits/sec  43598
[  5]  17.01-18.00  sec  59.6 MBytes   502 Mbits/sec  42808
[  5]  18.00-19.00  sec  59.6 MBytes   499 Mbits/sec  42828
[  5]  19.00-20.01  sec  59.5 MBytes   498 Mbits/sec  42719
```

Figura 5.1: Creación del cliente iperf3 para envio de datos, donde se envia trafico UDP a traves de una red MPLS: iperf3 -c 20.20.20.1 -u -b 500M -t 3600 -i 1

```
alan@alan-hplaptop15dy2xxx:~$ iperf3 -s
-----------------------------------------------------------
Server listening on 5201 (test #1)
-----------------------------------------------------------
Accepted connection from 10.10.10.1, port 53492
[  5] local 20.20.20.1 port 5201 connected to 10.10.10.1 port 59766
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-1.00   sec  58.9 MBytes   494 Mbits/sec  0.020 ms  0/42320 (0%)
[  5]   1.00-2.00   sec  59.2 MBytes   497 Mbits/sec  0.019 ms  0/42531 (0%)
[  5]   2.00-3.00   sec  59.6 MBytes   500 Mbits/sec  0.039 ms  0/42800 (0%)
[  5]   3.00-4.00   sec  59.5 MBytes   499 Mbits/sec  0.023 ms  0/42704 (0%)
[  5]   4.00-5.00   sec  59.7 MBytes   500 Mbits/sec  0.021 ms  0/42845 (0%)
[  5]   5.00-6.00   sec  60.0 MBytes   503 Mbits/sec  0.015 ms  0/43057 (0%)
[  5]   6.00-7.00   sec  59.3 MBytes   497 Mbits/sec  0.019 ms  0/42584 (0%)
[  5]   7.00-8.00   sec  59.8 MBytes   501 Mbits/sec  0.026 ms  0/42930 (0%)
[  5]   8.00-9.00   sec  59.6 MBytes   500 Mbits/sec  0.016 ms  0/42798 (0%)
[  5]   9.00-10.00  sec  59.6 MBytes   501 Mbits/sec  0.018 ms  0/42840 (0%)
```

Figura 5.2: Prueba de tráfico:levantamiento servidor iperf3 -s

Netdata cloud genera alertas automáticas, enviando por correo a los administradores, cualquier cambio de la red.

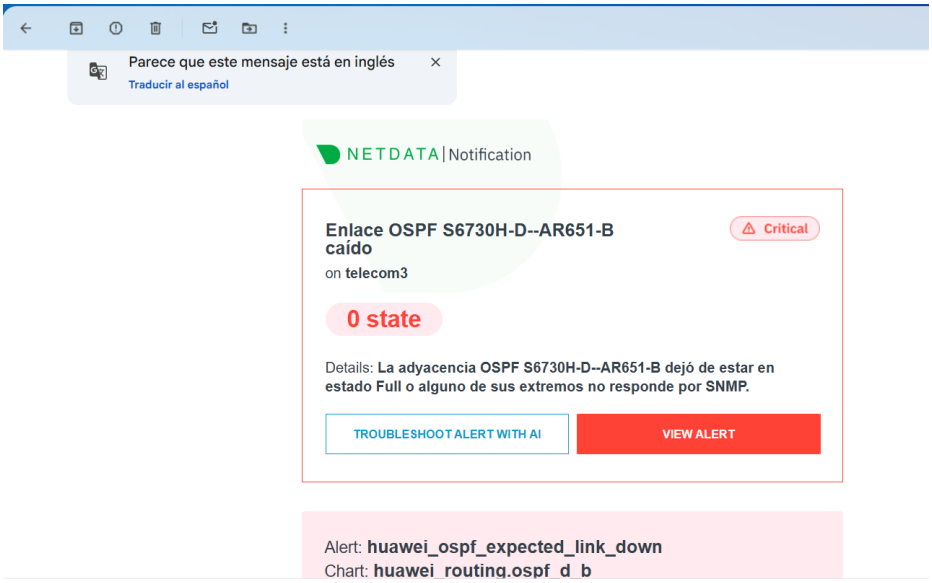


Figura 5.3: Notificación por E-mail automáticamente de Netdata

Bibliografía

[1] Huawei Technologies Co., Ltd., *Basic Experiment on Network Device Management*, Instructor Version. Guía de laboratorio; año indicado en el aviso de copyright, 2020.

[2] *Lab: MPLS Basics on Huawei AR651 Routers and S6730H Switches*, Student Lab Guide. Guía de laboratorio sin autor ni fecha indicados.

[3] Huawei Technologies Co., Ltd. «SNMP Configuration Commands - NetEngine AR600, AR6100, AR6200, and AR6300 V300R021 Command reference,» visitado 14 de oct. de 2026. dirección: https://support.huawei.com/enterprise/en/doc/EDOC1100203224/dab469bd/snmp-configuration-commands

[4] Netdata, Inc. «Configuration: Device Metrics,» Learn Netdata, visitado 14 de oct. de 2026. dirección: https://learn.netdata.cloud/docs/network-performance-monitoring/device-metrics/configuration

[5] Huawei Technologies Co., Ltd., *Configuring Automatic Enterprise Network O&M*, Student Version. Guía de laboratorio, sin fecha.

[6] S. D. Tórtola Bardales, L. A. Luna Hernández, E. J. Yes Morales y A. S. Gomez, «Lab 14: Integrando OSPF + SSH + Paramiko,» Universidad del Valle de Guatemala, Informe de laboratorio, 13 de mayo de 2026, Sistemas de Telecomunicaciones 2, sección 11.

[7] V. Gueant. «iPerf - Download iPerf3 and original iPerf pre-compiled binaries,» visitado 14 de oct. de 2026. dirección: https://iperf.fr/iperf-download.php

[8] Energy Sciences Network (ESnet). «Invoking iperf3,» visitado 14 de oct. de 2026. dirección: https://software.es.net/iperf/invoking.html

[9] Microsoft. «New-NetIPAddress (NetTCPIP),» visitado 14 de oct. de 2026. dirección: https://learn.microsoft.com/en-us/powershell/module/nettcpip/new-netipaddress?view=windowsserver2025-ps

[10] Microsoft. «Find-NetRoute (NetTCPIP),» visitado 14 de oct. de 2026. dirección: https://learn.microsoft.com/en-us/powershell/module/nettcpip/find-netroute?view=windowsserver2025-ps

[11] Microsoft. «New-NetRoute (NetTCPIP),» visitado 14 de oct. de 2026. dirección: https://learn.microsoft.com/en-us/powershell/module/nettcpip/new-netroute?view=windowsserver2025-ps

[12] Huawei Technologies Co., Ltd. «(Optional) Configuring a Policy for Triggering LDP LSP Establishment,» visitado 14 de oct. de 2026. dirección: https://support.huawei.com/enterprise/en/doc/EDOC1100198643/dba99bc3/optional-configuring-a-policy-for-triggering-ldp-lsp-establishment

[13] Huawei Technologies Co., Ltd. «lsp-trigger (IPv4 view),» visitado 14 de oct. de 2026. dirección: https://info.support.huawei.com/hedex/api/pages/EDOC1100277650/AZM1016J/04/resources/command/yunshan/CONFIGV4EGRESSPOLICY%28LDPP2POM%29.html

[14] Microsoft. «install command (WinGet),» visitado 14 de oct. de 2026. dirección: https://learn.microsoft.com/en-us/windows/package-manager/winget/install

[15] Microsoft. «New-NetFirewallRule (NetSecurity),» visitado 14 de oct. de 2026. dirección: https://learn.microsoft.com/en-us/powershell/module/netsecurity/new-netfirewallrule?view=windowsserver2025-ps

[16] Netdata, Inc. «Dashboards Tab,» visitado 21 de oct. de 2026. dirección: https://learn.netdata.cloud/docs/dashboards-and-charts/tabs/dashboards

[17] Netdata, Inc. «Creating Alerts with Netdata Alerts Configuration Manager,» visitado 21 de oct. de 2026. dirección: https://learn.netdata.cloud/docs/alerts-&-notifications/creating-alerts-with-the-alerts-configuration-manager

[18] Netdata, Inc. «SNMP Monitoring Guide: How To Monitor Network Devices,» visitado 21 de oct. de 2026. dirección: https://www.netdata.cloud/academy/snmp-monitoring-guide/

[19] Netdata, Inc. «Discovery Methods,» visitado 21 de oct. de 2026. dirección: https://learn.netdata.cloud/docs/network-performance-monitoring/topologies/discovery-methods

[20] Netdata. «OSPF Adjacency Topology,» visitado 21 de oct. de 2026. dirección: https://learn.netdata.cloud/docs/network-performance-monitoring/topologies/integrations/ospf-adjacency-topology

[21] MIB Browser. «HUAWEI-OSPFV2-MIB,» visitado 21 de oct. de 2026. dirección: https://mibs.observium.org/mib/HUAWEI-OSPFV2-MIB/

[22] Netdata, Inc. «SNMP Profile Format,» visitado 25 de ago. de 2026. dirección: https://learn.netdata.cloud/docs/network-performance-monitoring/device-metrics/configuration/snmp-profile-format